



Text Preprocessing

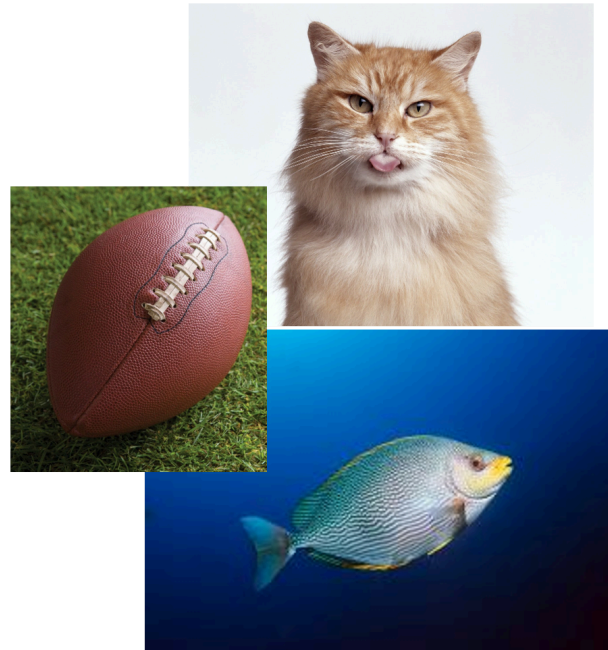
Filippo Chiarello & Vito Giordano

Intro to Textual Data - Data Types

Tabular Data

	age (n)	job (c)	marital (c)	education (c)	balance (n)	housing (c)
0	30	unemployed	married	primary	1787	no
1	33	services	married	secondary	4789	yes
2	35	management	single	tertiary	1350	yes
3	30	management	married	tertiary	1476	yes
4	59	blue-collar	married	secondary	0	yes
5	35	management	single	tertiary	747	no

Images



Text

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gef-burn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Intro to Textual Data - Data Types

Tabular Data	Images	Text
Structure: organized in rows and columns with clear relationships between variables. Easy to manipulate and analyze using statistical methods. Not capture complex relationships or unstructured information	Structure: consists of pixels arranged in a grid, often represented as arrays of numerical values. Rich source of information for visual recognition and interpretation. High-dimensional data can be resource-intensive to store and process.	Structure: Unstructured or semi-structured data in the form of words, sentences, and paragraphs. Rich source of information for understanding human language. Requires algorithms to process and capture context and semantics.

Intro to Textual Data - Natural Language Processing (NLP)

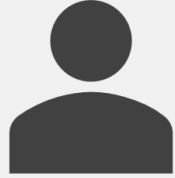
- A sub-field of Artificial Intelligence that studies how to computationally decode information in natural language.
- Main aim: bridge the gap in communication between computers and humans.
- NLP is the convergence between Linguistics and Computer science.

Linguistic studies:

TABLE 1.1: Some subfields of linguistics, moving from smaller structures to broader structures

Linguistics subfield	What does it focus on?
Phonetics	Sounds that people use in language
Phonology	Systems of sounds in particular languages
Morphology	How words are formed
Syntax	How sentences are formed from words
Semantics	What sentences mean
Pragmatics	How language is used in context

What Natural language is

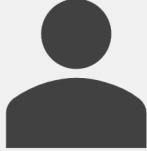


Natural Language refers to the type of language that **human beings** use to communicate with one another, such as English, Spanish, or Chinese. These languages are **complex**, constantly evolving, and rich in nuances, idiomatic expressions, and contextual meanings, which are easy for people to understand but **difficult for machines** to process.

1010
1010

Machine Language is the most basic language used by computers to perform operations. It consists of binary code, a series of 0s and 1s, which directly corresponds to the machine's hardware instructions. Unlike natural language, **machine language is rigid, unambiguous**, and lacks the nuances found in human languages.

What Natural language is

		1010 1010
Purpose	Human communication	Computer operation
Complexity	High, with flexible structure	Low, highly structured
Ambiguity	Contains idioms and multiple meanings	No ambiguity, strictly defined commands
Example	<i>"Can you help me?"</i>	<i>11010010101</i> (binary instruction)

Challenges of Natural Language

Ambiguity: Words and sentences can have multiple meanings depending on context (e.g., "bank" could mean a financial institution or the side of a river).

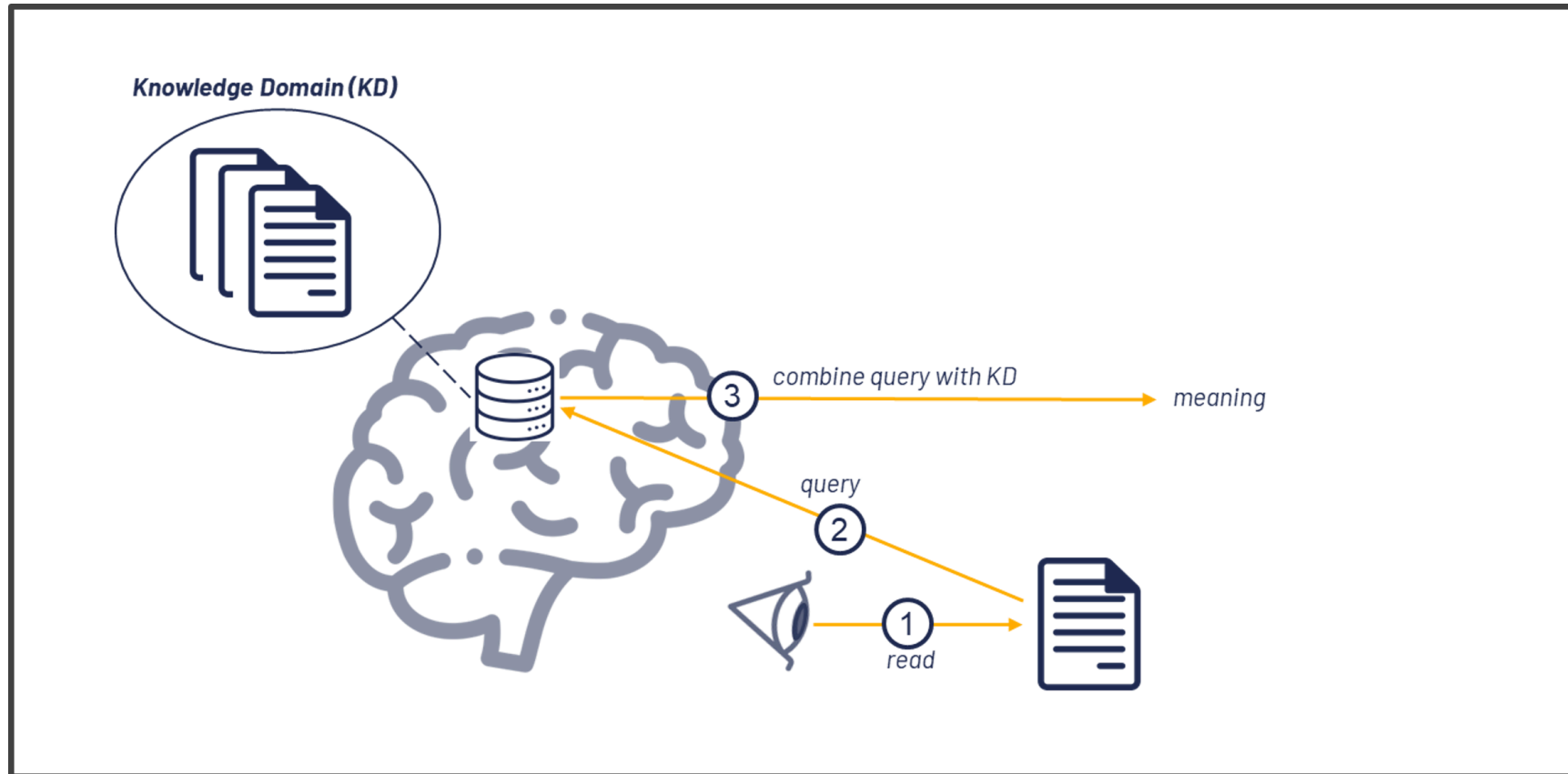
To fully understand a sentence or a word, one often needs **common sense knowledge** or **specific domain knowledge** to provide the right context. This context helps clarify meaning, resolve ambiguities, and understand implied information. However, this type of knowledge—whether common sense or specialized—**is not contained in the text** itself but exists only in the mind of the person reading it.

For example, understanding phrases like "break the ice" requires common sense to interpret it as "start a conversation," not as a literal action. Similarly, understanding a term like "ROI" (Return on Investment) demands knowledge of finance, a specific domain that the reader must be familiar with to grasp the term's significance in context.

When the airbag is deployed, the gas pumped into the airbag unfolds the airbag.

Challenges of Natural Language

Textual documents are **NOT self-contained** and they assume a certain level of background knowledge for their complete interpretation.



Intro to Textual Data - Textual Data Everywhere!

United States Patent [19] Oliver et al.		[11] Patent Number: 5,723,765 [45] Date of Patent: Mar. 3, 1998
[54] CONTROL OF PLANT GENE EXPRESSION		
[75] Inventors: Melvin John Oliver, Lubbock; Jerry Edwin Quisenberry, Idaho; Norma Lee Glover Trollinger, Quana, all of Tex.; Don Lee Keim, Leland, Miss.		
[73] Assignees: Delta and Pine Land Co., Scott, Miss.; The United States of America as represented by the Secretary of Agriculture, Washington, D.C.		
[21] Appl. No.: 477,559		
[22] Filed: Jun. 7, 1995		
Related U.S. Application Data		
[63] Continuation-in-part of Ser. No. 283,504, Aug. 1, 1994, abandoned.		
[51] Int. Cl.⁵ C12N 15/29; C12N 15/82; A01H 4/00; A01H 5/00		
[52] U.S. Cl. 800/205; 800/250; 536/24.1; 536/23.6; 536/24.5; 435/320.1; 435/240.4; 435/172.3		
[58] Field of Search 536/24.1; 23.6; 536/24.5; 435/320.1; 240.4; 172.3; 800/205; 250		
[56] References Cited		
U.S. PATENT DOCUMENTS		
4,999,317	9/1990	Sauer 435/172.3
5,159,135	10/1992	Unbeck 800/205
5,217,889	6/1993	Robinson et al. 435/172.3
5,231,019	7/1993	Paszowski et al. 435/172.3
5,244,802	9/1993	Rangan 435/240.5
5,270,201	12/1993	Richards et al. 435/240.4
5,304,730	4/1994	Lawson et al. 800/205
5,352,605	10/1994	Finley et al. 435/240.4

Lanzer, Michael and Bujard, Hermann, "Promoters largely determine the efficiency of repressor action," *Proc. Natl. Acad. Sci. USA*, 85: 8973-8977 (1988).

Araki, Kimi et al., *Site-specific recombination of a transgene in fertilized eggs by transient expression of Cre recombinase*, *Proc. Natl. Acad. Sci. USA*, 92(1):160-164 (1995).

Modberry, Scott L., et al., *Intra-chromosomal rearrangements generated by Cre-lox site-specific recombination*, *Nucleic Acids Research*, 23(3):485-490 (1995).

Chapman, Sean, Kavanagh, Tony and Baulcombe, David, "Potato virus X as a vector for gene expression in plants," *The Plant Journal*, 2(4):549-557 (1992).

Odell, Joan T., et al., *Seed-Specific Gene Activation Mediated by the Cre/lox Site-Specific Recombination System*, *Plant Physiol.*, 106:447-458 (Oct. 2, 1994).

Qin, Minmin, et al., *Cre recombinase-mediated site-specific recombination between plant chromosomes*, *Proc. Natl. Acad. Sci. USA*, 91:1706-1710 (1994).

Sauer, Brian, *Manipulation of Transgenes by Site-Specific Recombination: Use of Cre Recombinase*, *Methods in Enzymology*, 225:890-900 (1993).

Barinaga, Marcia, "Knockout mice: round two," *Science*, 265:26-28 (1994).

Gu, Hua, Marth, Jamey D, Orban, Paul C., Mottmann, Horst and Rajewsky, Klaus, "Deletion of a DNA of a polymerase B gene segment in T cells using cell type-specific gene targeting," *Science*, 265:103-106 (1994).

(List continued on next page.)

Primary Examiner—Douglas W. Robinson
Assistant Examiner—Thomas Haas
Attorney, Agent, or Firm—Rodwell, Figg, Ernst & Kurz

[57] ABSTRACT

A method for making a genetically modified plant comprising regenerating a whole plant from a plant cell that has been transfected with DNA sequences comprising a first gene

55 Claims, No Drawings



Contents lists available at [ScienceDirect](#)

World Patent Information

journal homepage: [www.elsevier.com/locate/worpatin](#)

Evaluating generative patent language models

Jieh-Sheng Lee

National Yang Ming Chiao Tung University (NYCU) School of Law, No. 1001, Daxue Rd., Hsinchu, 300093, Taiwan

ARTICLE INFO

Keywords:
Natural language generation
Natural language processing
Patent text generation
Deep learning

ABSTRACT

Generative language models are promising for assisting human writing in various domains. This manuscript aims to build generative language models in the patent domain and evaluate model performance from a human-centric perspective. The perspective is to measure the ratio of keystrokes that can be saved by auto-completion based on generative patent language models. A higher ratio means a more effective model which can save more keystrokes. This metric can be used to benchmark model performance. The metric is keystroke-based and different from conventional machine-centric metrics that are token-based. In terms of model size, the largest model built in this manuscript is PatentGPT-1.6B, which is state-of-the-art in the patent domain. Based on the metric, it is found that the largest model is not necessarily the best for the human-centric metric. The finding means that keeping increasing model sizes in the patent domain might be unnecessary if the purpose is to assist human writing with auto-completion. Several patent language models are pre-trained from scratch in this research. The pre-trained models are released for future researchers. Several visualization tools are also provided. The importance of building a generative language model in the patent domain is its potential to facilitate creativity and innovations in the future.

1. Introduction

Generative language models have shown great potential in recent years. To train a generative model, it is common to collect a large amount of general or diverse data and train a model to generate similar data. The mainstream research is to continue to build larger models with more data. The evaluation of generative language models is also machine-centric and token-based. In this manuscript, instead toward the goal. In addition, the performance of the models in different sizes can be evaluated based on the metric. New patents are granted frequently and are available to the public for free. By measuring a number of patents collectively with the metric, it is possible to measure which model performs best.

The major contributions of this manuscript include: (1) pre-trained several language models specific to the patent domain; (2) proposed a human-centric metric for model evaluation (how many keystrokes to be

 **Twitter Support** 
@TwitterSupport

We'll soon be implementing some changes in our effort to reduce spam in Direct Messages. Unverified accounts will have daily limits on the number of DMs they can send. Subscribe today to send more messages: [twitter.com/i/twitter_blue...](#)

2:22 PM · Jul 21, 2023 · 329.4K Views



United Nations

S/2019/822

Distr.: General

15 October 2019

Original: English

Central African Republic
Report of the Secretary-General

I. Introduction

1. By its resolution 2448 (2019), the Security Council extended the mandate of the United Nations Multidimensional Integrated Stabilization Mission in the Central African Republic (MINUSCA) until 15 November 2019 and requested me to report on its implementation on a quarterly basis. The present report provides an update on major developments in the Central African Republic since my report of 17 June (S/2019/498) and recommendations on the extension of the mandate of MINUSCA.

II. Political situation

2. Progress was made in implementing the Political Agreement for Peace and Reconciliation in the Central African Republic, with no noticeable efforts by the Government, including the establishment of its follow-up mechanisms. Security incidents and human rights violations linked to the conflict decreased. However, significant challenges remain, including abuses against civilians and violent clashes between signatory armed groups. Popular distrust of the Agreement, in particular within the political opposition, continued as crucial preparations began for the presidential, legislative and local elections scheduled for 2020 and 2021, integral to an inclusive political process, indispensable to consolidate democracy and vital to ensure political stability.

Peace process

3. For the first time since the signing of the Agreement, the Government and 13 out of 14 signatory armed groups met in Bangui on 23 and 24 August, co-chaired by the Prime Minister, Faustin Armand Tsiranigba, and the Special Representative and Head of Office for the African Union Office in the Central African Republic, My Special Representative for the Central African Republic and Head of MINUSCA, Mandavisa Ndlovu, and representatives of the Economic Community of Central African States (ECCAS), Angola, Cameroon, Chad, the Democratic Republic of Congo and Equatorial Guinea participated. The parties agreed, inter alia, to cease violence against civilians, expedite cessation of hostilities and restoration of State authority and improve communication. They underscored the need to sanction non-compliant signatories.

2448 (E) - 2110 19

2448 (E) - 2110 19



Please recycle



Alice L wrote a review Feb 2022

Horseheads, New York • 44 contributions • 17 helpful votes



Perfect for Us

"We recently chose Motto for an overnight in NYC and were so glad we did. Our stay started off with a warm and friendly greeting from the staff at the front desk. Check in was easy and efficient. We found our room to be clean and comfortable- brand new! Quite beautiful. The view from our room as the sun was setting was spectacular! After an evening at the theatre (I highly recommend The Music Man!), the bar was open for a night cap. Breakfast was fresh and delicious served by more friendly staff. Our overall experience was exceptional! We'll be back"

Read less ▲

Hotel's Favourite

 Draft Saved

 Eki Priscilla Ayemwe
Draft

Title

7 reasons why a blog makes your business more successful

A Business blog is a powerful marketing channel when integrated into your business strategy. It's both a business hub and a business driver.

However, many business owners are yet to take advantage of the benefits available through business blogging.

Many businesses have business websites that only start to get business many years after creation. This is because the pages are not updated regularly to drive traffic and get traffic to your business.

Ready to publish?

Share draft link

Add to publication

See revision history

Share to Twitter

Manage unfilled setting

Manage content licensing

Delete story

Change featured image

Change display title / subtitle

Change tags

Customize story link

Hints and keyboard shortcuts

More help

How Can We Describe a Text?

- **Length of the Document:** Number of words, number of sentences, and overall size.
- **Syntactic Complexity:** The structure of sentences, including grammar, sentence length, and use of complex sentence forms.
- **Semantic Complexity:** Depth of meaning, abstract concepts, and nuanced language within the text.
- **Frequency of Mistakes:** Spelling, grammar, and typographical errors, which can impact readability and clarity.
- **Formality Level:** Whether the language is formal or informal, often guided by conventions and tone.
- **Document Accessibility:** Ease of understanding, which may involve readability levels and clear language.
- **Document Language:** The language in which the document is written, impacting cultural context and comprehension.

How Can We Describe a Text?

These factors are influenced by:

- **Purpose of the Writing:** The goal of the document (e.g., instructional, persuasive, informative).
- **Writer's Background:** Expertise, language proficiency, and perspective of the author.
- **Audience:** Characteristics of the intended readers, including their familiarity with the topic and language.

Intro to Textual Data - Textual Data Everywhere!

United States Patent [19] Oliver et al.		[11] Patent Number: 5,723,765 [45] Date of Patent: Mar. 3, 1998
[54] CONTROL OF PLANT GENE EXPRESSION		
[75] Inventors: Melvin John Oliver, Lubbock; Jerry Edwin Quisenberry, Idaho; Norma Lee Glover Trollinger, Quana, all of Tex.; Don Lee Keim, Leland, Miss.		
[73] Assignees: Delta and Pine Land Co., Scott, Miss.; The United States of America as represented by the Secretary of Agriculture, Washington, D.C.		
[21] Appl. No.: 477,559		
[22] Filed: Jun. 7, 1995		
Related U.S. Application Data		
[63] Continuation-in-part of Ser. No. 283,504, Aug. 1, 1994, abandoned.		
[51] Int. Cl.⁵ C12N 15/29; C12N 15/82; A01H 4/00; A01H 5/00		
[52] U.S. Cl. 800/205; 800/250; 536/24.1; 536/23.6; 536/24.5; 435/320.1; 435/240.4; 435/172.3		
[58] Field of Search 536/24.1; 23.6; 536/24.5; 435/320.1; 240.4; 172.3; 800/205; 250		
[56] References Cited		
U.S. PATENT DOCUMENTS		
4,999,317 9/1990 Sauer 435/172.3 5,159,135 10/1992 Unbeck 800/205 5,217,889 6/1993 Robinson et al. 435/172.3 5,231,019 7/1993 Paszkowski et al. 435/172.3 5,244,802 9/1993 Rangan 435/240.5 5,270,201 12/1993 Richards et al. 435/240.4 5,304,730 4/1994 Lawson et al. 800/205 5,352,605 10/1994 Finley et al. 435/240.4		
55 Claims, No Drawings		

 Alice L wrote a review Feb 2022
Horseheads, New York • 44 contributions • 17 helpful votes



Perfect for Us

"We recently chose Motto for an overnight in NYC and were so glad we did. Our stay started off with a warm and friendly greeting from the staff at the front desk. Check in was easy and efficient. We found our room to be clean and comfortable- brand new! Quite beautiful. The view from our room as the sun was setting was spectacular! After an evening at the theatre (I highly recommend The Music Man!), the bar was open for a night cap. Breakfast was fresh and delicious served by more friendly staff. Our overall experience was exceptional! We'll be back"

Read less ▲



Contents lists available at [ScienceDirect](#)

World Patent Information

journal homepage: [www.elsevier.com/locate/worpatin](#)

Evaluating generative patent language models

Jieh-Sheng Lee

National Yang Ming Chiao Tung University (NYCU) School of Law, No. 1001, Daxue Rd., Hsinchu, 300093, Taiwan

ARTICLE INFO

ABSTRACT

Keywords:
Natural language generation
Natural language processing
Patent text generation
Deep learning

Generative language models are promising for assisting human writing in various domains. This manuscript aims to build generative language models in the patent domain and evaluate model performance from a human-centric perspective. The perspective is to measure the ratio of keystrokes that can be saved by auto-completion based on generative patent language models. A higher ratio means a more effective model which can save more keystrokes. This metric can be used to benchmark model performance. The metric is keystroke-based and different from conventional machine-centric metrics that are token-based. In terms of model size, the largest model built in this manuscript is PatentGPT-14B, which is state-of-the-art in the patent domain. Based on the metric, it is found that the largest model is not necessarily the best for the human-centric metric. The finding means that keeping increasing model sizes in the patent domain might be unnecessary if the purpose is to assist human writing with auto-completion. Several patent language models are pre-trained from scratch in this research. The pre-trained models are released for future researchers. Several visualization tools are also provided. The importance of building a generative language model in the patent domain is its potential to facilitate creativity and innovations in the future.

1. Introduction

Generative language models have shown great potential in recent years. To train a generative model, it is common to collect a large amount of general or diverse data and train a model to generate similar data. The mainstream research is to continue to build larger models with more data. The evaluation of generative language models is also machine-centric and token-based. In this manuscript, instead

toward the goal. In addition, the performance of the models in different sizes can be evaluated based on the metric. New patents are granted frequently and are available to the public for free. By measuring a number of patents collectively with the metric, it is possible to measure which model performs best.

The major contributions of this manuscript include: (1) pre-trained several language models specific to the patent domain; (2) proposed a human-centric metric for model evaluation (how many keystrokes to be



Twitter Support ✓
@TwitterSupport

We'll soon be implementing some changes in our effort to reduce spam in Direct Messages. Unverified accounts will have daily limits on the number of DMs they can send. Subscribe today to send more messages: [twitter.com/i/twitter_blue...](#)

2:22 PM · Jul 21, 2023 · **329.4K** Views

United Nations



Security Council

S/2019/822

Dist.: General
15 October 2019
Original: English

Central African Republic

Report of the Secretary-General

I. Introduction

1. By its resolution 2448 (2019), the Security Council extended the mandate of the United Nations Multidimensional Integrated Stabilization Mission in the Central African Republic (MINUSCA) until 15 November 2019 and requested me to report on its implementation on a quarterly basis. The present report provides an update on major developments in the Central African Republic since my report of 17 June (S/2019/498) and recommendations on the extension of the mandate of MINUSCA.

II. Political situation

2. Progress was made in implementing the Political Agreement for Peace and Reconciliation in the Central African Republic, with no moderate efforts by the Government, including the establishment of its follow-up mechanisms. Security incidents and human rights violations linked to the conflict decreased. However, significant challenges remain, including abuses against civilians and violent clashes between signatory armed groups. Popular distrust of the Agreement, in particular within the political opposition, continued as crucial preparations began for the presidential, legislative and local elections scheduled for 2020 and 2021, integral to an inclusive political process, indispensable to consolidate democracy and vital to ensure political stability.

Peace process

3. For the first time since the signing of the Agreement, the Government and 13 out of 14 signatory armed groups met in Bangui on 23 and 24 August, co-chaired by the Prime Minister, Faustin Armand Tsedze, and the Special Representative and Head of Office for the African Union Office in the Central African Republic, My Special Representative for the Central African Republic and Head of MINUSCA, Mandavisa Ndlovu, and representatives of the Economic Community of Central African States (ECCAS), Angola, Cameroon, Chad, the Democratic Republic of Congo and Equatorial Guinea participated. The parties agreed, inter alia, to cease violence against civilians, expedite cessation of hostilities and restoration of State authority and improve communication. They underscored the need to sanction non-compliant signatories.

2448 (E) - 2110 19



Please recycle 



M Draft Saved

Ready to publish?

Share draft link
Add to publication
See revision history
Share to Twitter
Manage unified setting
Manage content licensing
Delete story
Change featured image
Change display title / subtitle
Change tags
Customize story link
Hints and keyboard shortcuts
More help

Title

7 reasons why a blog makes your business more successful

A Business blog is a powerful marketing channel when integrated into your business strategy. It's both a business hub and a business driver.

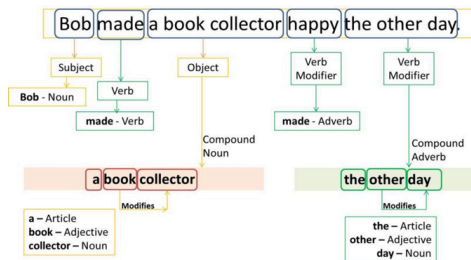
However, many business owners are yet to take advantage of the benefits available through business blogging.

Many businesses have business websites that only start to get business many years after creation. This is because the pages are not updated regularly to drive traffic and get traffic to your business.

Intro to Textual Data - The Pipeline

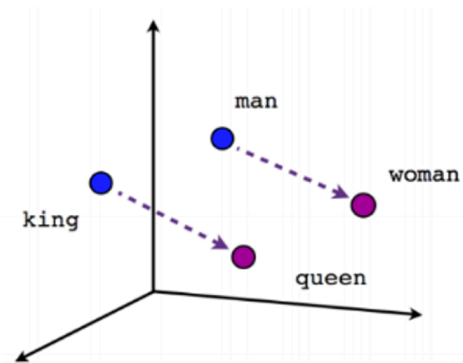
Text Preprocessing

This phase focuses on cleaning and standardizing the text to **reduce noise**. By removing irrelevant elements and harmonizing formats, preprocessing improves data quality and consistency. In addition, it helps manage the high dimensionality of textual data.



Text Representation

Text representation refers to the process of converting raw textual data into structured, **machine-readable formats** that can be analyzed computationally.



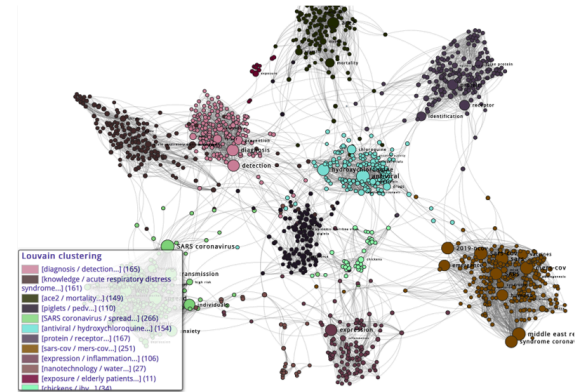
Text Modelling

Text modelling uses computational methods to extract information from text. It can be **supervised**, when models learn from labeled data to identify predefined categories or constructs, or **unsupervised**, when models explore the text to uncover patterns such as topics or clusters without prior labels.

tourists, the **third ORDINAL** most visited in **the European Union ORG**, and the most popular tourist destination. [13] Its historic centre is listed by **UNESCO ORG** as a **World Heritage ORG** Site. [14] Host city for the **Summer Olympics EVENT**, **Rome GPE** is also the seat of several specialised agencies of **the United ORG**, such as **the Food and Agriculture Organization ORG** (**FAO ORG**), **the World Food Programme ORG** (**WFP ORG**) and **the International Fund for Agricultural Development ORG** (**IFAD ORG**). The city also hosts the Secretariat of **the Parliamentary Assembly ORG** of the **Union for the Mediterranean [15]** (**UfM ORG**) as headquarters of many international business companies such as **Eni ORG**, **Enel GPE**, **TIM ORG**, **S.p.A. ORG**, and national and international banks such as **Unicredit ORG** and **BNL ORG**. **Rome GPE** business district is the home of many companies involved in the oil industry, the pharmaceutical industry, services. The presence of renowned international brands in the city have made **Rome GPE** an important center for business.

Statistical Analysis

Outputs of Text modelling are subsequently used as inputs for **downstream analyses**.





Text Pre-processing

Text pre-processing allows us to **manipulate**, **summarize**, and **visualize** the characteristics of raw text easily for Natural Language Processing (NLP) task.

Pre-processing Phases

There are many operations that can be performed to pre-process a text. The following phases are the most commonly employed:

1. Tokenization
2. Text Cleaning (Stop words Removal)
3. Part-of-Speech (POS) tagging
4. Stemming
5. Lemmatization

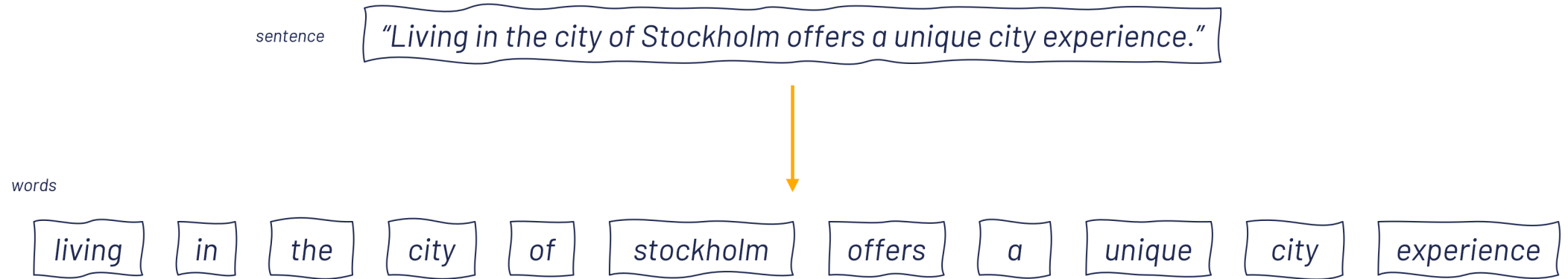


Tokenization

Tokenization - What is it?

Tokenization is the process of **splitting text into tokens**.

A **token** is a meaningful unit of text, such as a word, a sentence, or paragraph, that we are interested in using for analysis.



Tokenization - How to perform it?

- `tidytext` is an R package for analysing text with the `tidyverse` philosophy
- treating **text as data frames** of individual words allows us to manipulate, summarize, and visualize the characteristics of text easily and integrate natural language processing into effective workflows of the tidyverse
- we define the tidy text format as being a table with **one-token-per-row**

```
# Install packages (only once)
install.packages(c("tidyverse", "tidytext"))

# Load Packages
library(tidyverse)
library(tidytext)
```

See more at: <https://smltar.com/tokenization.html#what-is-a-token>

unnest_tokens

- `unnest_tokens` is the main verb of tidytext
- it **splits** text into tokens and outputs a **one-token-per-row table**
- takes 3 main parameters:
 1. **tbl**: a data frame containing the text to tokenize
 2. **output**: the output column to be created
 3. **input**: the input column that gets split

What the function does:

- **punctuation** is stripped
- tokens are converted to **lowercase**
- **other columns**, such as the line number each word came from, are **retained**

unnest_tokens



```
# Load Packages
library(tidyverse)
library(tidytext)

# Define sentences
text <- c("living in the city of Stockholm offers a unique city experience.",
          "Rome is the capital of Italy.",
          "Pisa is a small city.")

# Create a table with one row per sentence
text_data <- tibble(Sentence_ID = 1:3, text = text)
text_data
```

```
## # A tibble: 3 × 2
##   Sentence_ID text
##   <int> <chr>
## 1      1 living in the city of Stockholm offers a unique ci...
## 2      2 Rome is the capital of Italy.
## 3      3 Pisa is a small city.
```


unnest_tokens



```
# Tokenize the selected sentence
text_data |>
  unnest_tokens(output = word, input = text)
```

```
## # A tibble: 22 × 2
##   Sentence_ID word
##         <int> <chr>
## 1           1 living
## 2           1 in
## 3           1 the
## 4           1 city
## 5           1 of
## 6           1 stockholm
## # i 16 more rows
```

Tokenizing Patent Data

Thanks to tokenization we can perform some exploratory analysis on textual data.
Let's do it on a **dataset of patents**.

Tokenizing Patent Data

The dataset contains **9,812 patents in the field of Artificial Intelligence (AI)** applications.

- The column **id** contains the publication numbers of patents;
- The column **title** contains the titles of patents;
- The column **abstract** contains the abstracts of patents.

id	title	abstract
EP1941301_B1	"SYSTEM AND METHODS FOR ENHANCING AN IMAGE.."	"A system and methods for enhancing an image (300)..."
EP2126563_B1	"METHOD FOR DETERMINING HEALTH STATUS BY ANALYZING ANALYTES"	"A method to extract information from samples taken at different times from a mammalian. These samples are..."
...	...	...

Tokenizing Patent Data

```
# Import a dataset of patents
patent_data <- read_rds("data/patents.rds")
patent_data
```

```
## # A tibble: 9,818 × 3
##   id          title          abstract
##   <chr>      <chr>          <chr>
## 1 EP1712182_B1 Method of ultrasonic detection and local... A metho...
## 2 EP1941301_B1 SYSTEM AND METHODS FOR ENHANCING AN IMAG... A syste...
## 3 EP2126563_B1 METHOD FOR DETERMINING HEALTH STATUS BY ... A metho...
## 4 EP2169547_B1 Compilation model for programmable logic... The cla...
## 5 EP2240878_B1 IDENTIFICATION AND VERIFICATION OF AN UN... Techniq...
## 6 EP2252889_B1 METHOD AND SYSTEM FOR ANALYSIS OF FLOW C... An auto...
## # i 9,812 more rows
```

Tokenizing Patent Data

```
# Tokenize patent abstracts
tokenized_abstracts <- patent_data |>
  unnest_tokens(output = word, input = abstract)
```

```
tokenized_abstracts
```

```
## # A tibble: 1,179,931 × 3
##   id          title                                word
##   <chr>        <chr>                                <chr>
## 1 EP1712182_B1 Method of ultrasonic detection and localiza... a
## 2 EP1712182_B1 Method of ultrasonic detection and localiza... meth...
## 3 EP1712182_B1 Method of ultrasonic detection and localiza... of
## 4 EP1712182_B1 Method of ultrasonic detection and localiza... ultr...
## 5 EP1712182_B1 Method of ultrasonic detection and localiza... dete...
## 6 EP1712182_B1 Method of ultrasonic detection and localiza... and
## # i 1,179,925 more rows
```

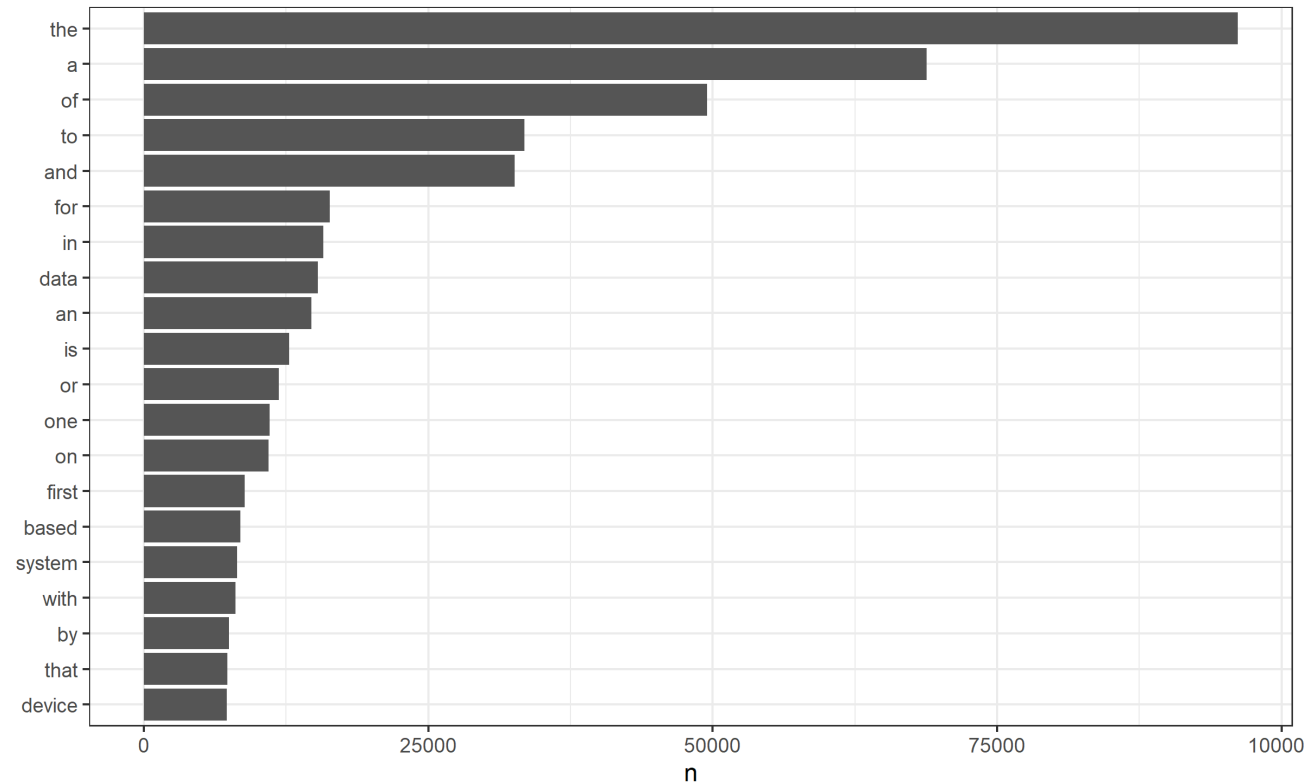
Tokenizing Patent Data - Word Frequency

Let's visualize the word frequency of words in patent abstracts.

```
#Plot the word frequency of the top 20 most common words in patent abstracts.
top_20_word_freq <- tokenized_abstracts |>
  count(word, sort = TRUE) |>
  top_n(n = 20) |>
  # reorder levels of factor word wrt n
  mutate(word = reorder(word, n)) |>
  ggplot(aes(word, n)) +
  geom_col() +
  xlab(NULL) +
  coord_flip() +
  theme_bw()
```

Tokenizing Patent Data - Word Frequency

Let's visualize the word frequency for the words of patent abstracts.

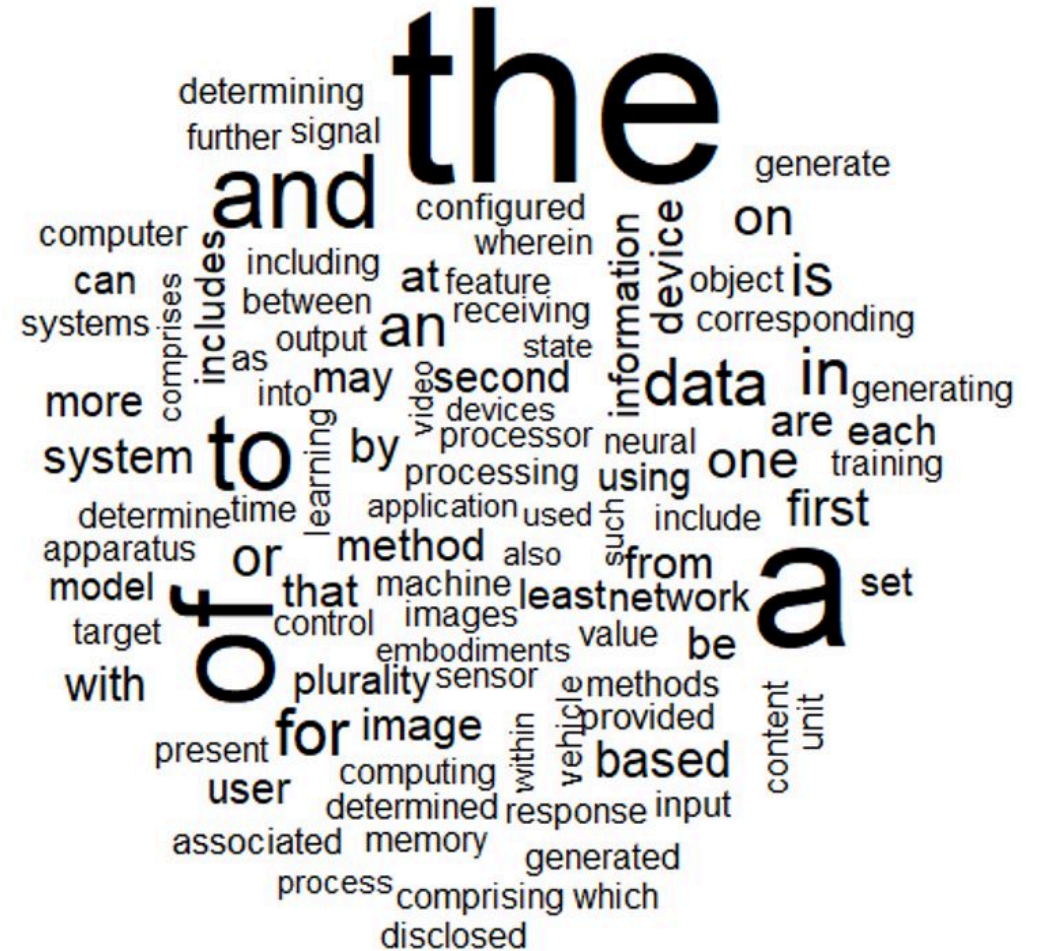


Tokenizing Patent Data - Word Cloud

Let's visualize the word cloud of words from patent abstracts

```
# Import Packages
library(wordcloud)

# Plot word cloud
tokenized_abstracts |>
  count(word) |>
  # evaluate an R expression in an environment
  with(wordcloud(word, n, max.words = 100))
```





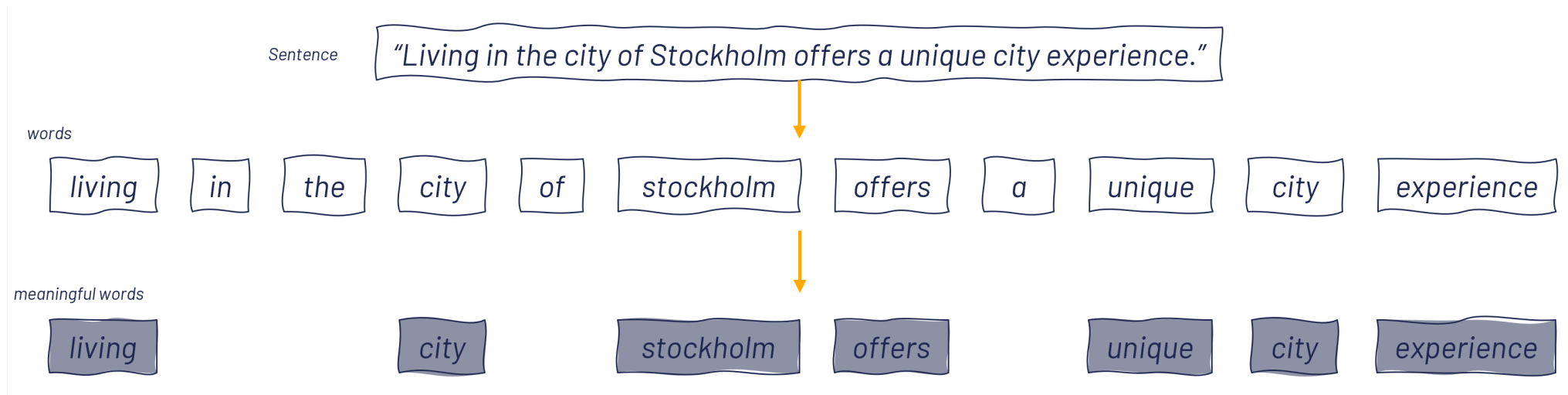
Text Cleaning

After tokenizing the text, useless or noisy tokens are usually eliminated. Numbers, symbols, typos, URLs and **stop words** do not add any semantic information.

Stop words

- **stop words** are the most commonly used words in languages. They are articles, prepositions, and other very frequent words that contribute little or nothing to the overall meaning of the text. Some examples: *the, is, at, which, for, an, on*.

Stop words removal is the process of **filtering out stop words**, to reduce the amount of noise in the textual data.



Stop words - Depend on Context

A quick option for using stop words is to get a list that has already been created. This is appealing because it is not difficult, but be aware that not all lists are created equal and **stop words depend on context.**

Stop words - Depend on Context

Stop words depend on:

- **Language:** different languages have different stop words (e.g., English: "the", "is", "in", "and"; French: "le", "est", "dans").
- **Domain:** In specific fields, the importance of certain words changes. For example, in AI field, words like "AI", "model", and "machine learning" might be stop words.
- **Document:** different documents have different common words. For example, patent documents often use legal phrases like "skilled in the art", "scope of invention", and "not limiting" that can be considered stop words since they do not provide any technical information.

To see more on stopwords: <https://smltar.com/stopwords.html#premadestopwords>

Stop words - Removing



Let's visualize a list of stopwords

```
# Visualize the stop words list of tidytext package
stop_words
```

```
## # A tibble: 1,149 × 2
##   word      lexicon
##   <chr>    <chr>
## 1 a       SMART
## 2 a's     SMART
## 3 able    SMART
## 4 about   SMART
## 5 above   SMART
## 6 according SMART
## # i 1,143 more rows
```

Stop words - Removing



```
# Define sentences
text <- c("living in the city of Stockholm offers a unique city experience.",
         "Rome is the capital of Italy.",
         "Pisa is a small city.")

# Create a table with one row per sentence
text_data <- tibble(Sentence_ID = 1:3, text = text)
text_data
```

```
## # A tibble: 3 × 2
##   Sentence_ID text
##   <int> <chr>
## 1      1 living in the city of Stockholm offers a unique ci...
## 2      2 Rome is the capital of Italy.
## 3      3 Pisa is a small city.
```

Stop words - Removing



```
# Tokenize the selected sentence
tokenized_sentence <- text_data |>
  unnest_tokens(output = word, input = text)
```

```
tokenized_sentence
```

```
## # A tibble: 22 × 2
##   Sentence_ID word
##         <int> <chr>
## 1           1 living
## 2           1 in
## 3           1 the
## 4           1 city
## 5           1 of
## 6           1 stockholm
## # i 16 more rows
```

Stop words - Removing

This can be achieved with an anti join, which returns all rows from the left table without a match in the right table.

```
# Remove stopwords from the tokenized sentence
tokenized_sentence |>
  anti_join(stop_words)
```

```
## # A tibble: 12 × 2
##   Sentence_ID word
##         <int> <chr>
## 1           1 living
## 2           1 city
## 3           1 stockholm
## 4           1 offers
## 5           1 unique
## 6           1 city
## # i 6 more rows
```


Stop words - Patent Data



Let's remove stopwords from patent abstracts

```
# Import patent data
patent_data <- read_rds("data/patents.rds")

# Tokenize patent abstracts
tokenized_abstracts <- patent_data |>
  unnest_tokens(output = word, input = abstract)

# Remove stopwords from abstracts
abstracts_without_stopwords <- tokenized_abstracts |>
  anti_join(stop_words)
```

Stop words - Patent Data



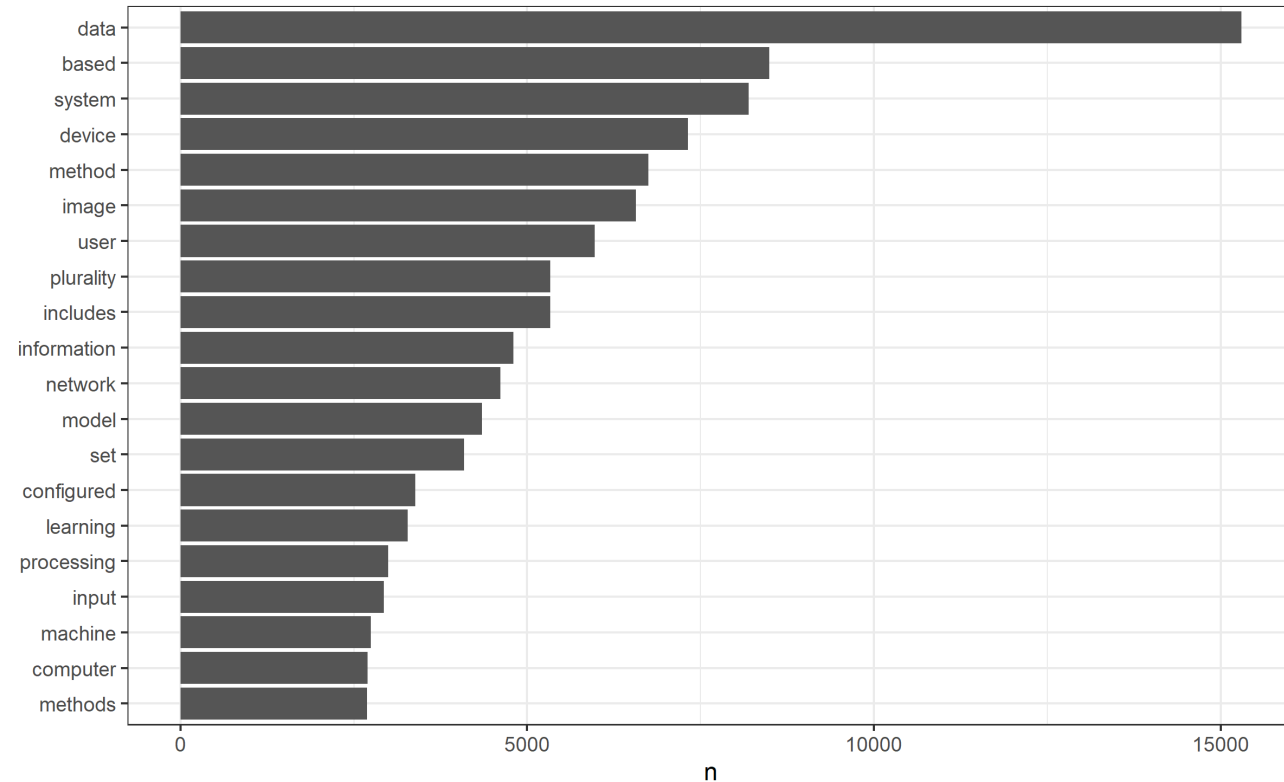
Now, let's visualize words frequency of patent abstracts **after removing stopwords**.

```
#Plot the word frequency of the top 20 most common words in patent abstracts.
top_20_word_freq <- abstracts_without_stopwords |>
  count(word, sort = TRUE) |>
  top_n(n = 20) |>
  # reorder levels of factor word wrt n
  mutate(word = reorder(word, n)) |>
  ggplot(aes(word, n)) +
  geom_col() +
  xlab(NULL) +
  coord_flip() +
  theme_bw()
```

Stop words - Patent Data



Now, let's visualize words frequency of patent abstracts **after removing stopwords**.



Bigrams Tokenization

Generate a list of bigrams from the text, splitting it into pairs of consecutive words.

```
library(rebus)

# Tokenize in bigrams
bigrams <- patent_data |>
  unnest_tokens(output = word, input = abstract, token = "ngrams", n = 2)

bigrams
```

```
## # A tibble: 1,170,136 × 3
##   id          title          word
##   <chr>      <chr>          <chr>
## 1 EP1712182_B1 Method of ultrasonic detection and localiza... a me...
## 2 EP1712182_B1 Method of ultrasonic detection and localiza... meth...
## 3 EP1712182_B1 Method of ultrasonic detection and localiza... of u...
## 4 EP1712182_B1 Method of ultrasonic detection and localiza... ultr...
## 5 EP1712182_B1 Method of ultrasonic detection and localiza... dete...
## 6 EP1712182_B1 Method of ultrasonic detection and localiza... and ...
## # i 1,170,130 more rows
```

Bigrams Tokenization

Then, remove bigrams containing stopwords.

```
stop_words_vec <- stop_words |>
  pull(word) |>
  unique()

regex_stopwords <- or1(str_c("\\b", stop_words_vec, "\\b"))

bigrams <- bigrams |>
  filter(!str_detect(word, regex_stopwords))

bigrams |> head(4)
```

```
## # A tibble: 4 × 3
##   id          title          word
##   <chr>      <chr>      <chr>
## 1 EP1712182_B1 Method of ultrasonic detection and localiza... ultr...
## 2 EP1712182_B1 Method of ultrasonic detection and localiza... cont...
## 3 EP1712182_B1 Method of ultrasonic detection and localiza... agen...
## 4 EP1712182_B1 Method of ultrasonic detection and localiza... rece...
```

Stop words - Patent Data



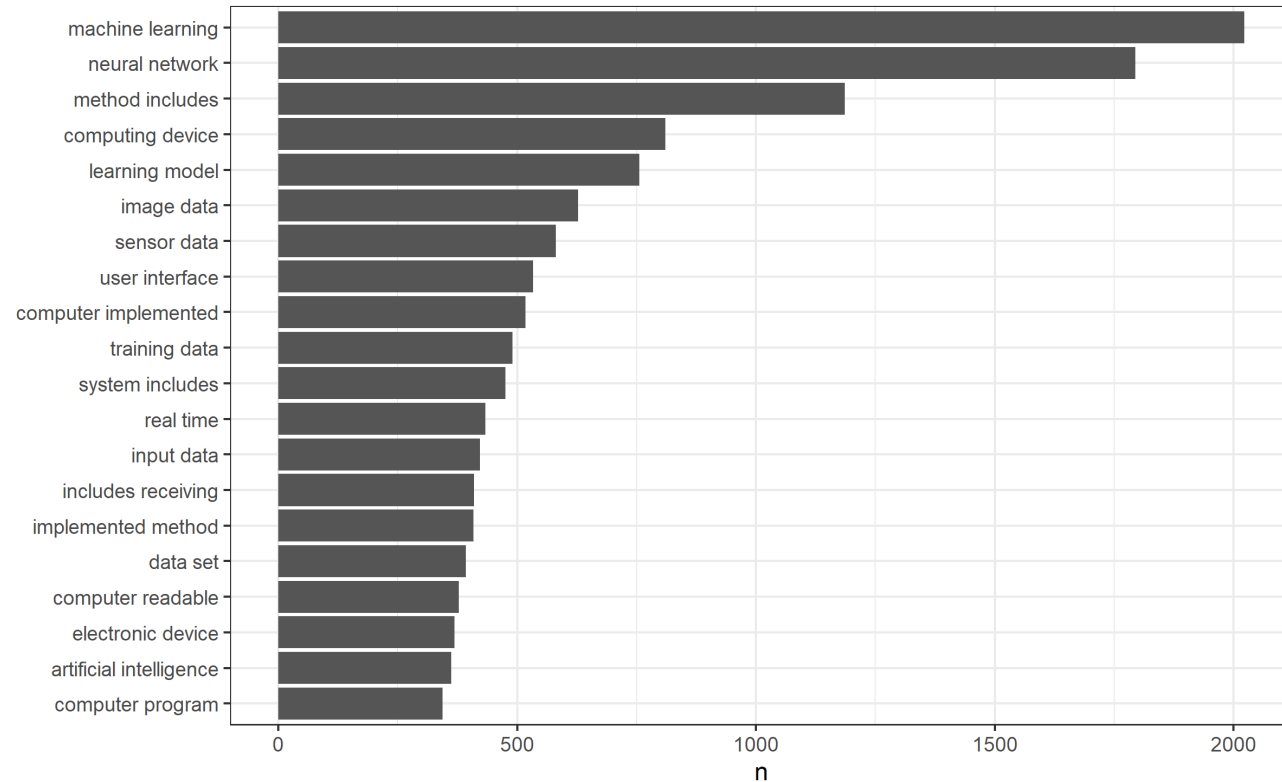
Now, let's visualize the frequency of bigrams from patent abstracts **after removing stopwords**.

```
#Plot the word frequency of the top 20 most common words in patent abstracts.
top_20_word_freq <- bigrams |>
  count(word, sort = TRUE) |>
  top_n(n = 20) |>
  # reorder levels of factor word wrt n
  mutate(word = reorder(word, n)) |>
  ggplot(aes(word, n)) +
  geom_col() +
  xlab(NULL) +
  coord_flip() +
  theme_bw()
```

Stop words - Patent Data



Now, let's visualize the frequency of bigrams from patent abstracts **after removing stopwords**.

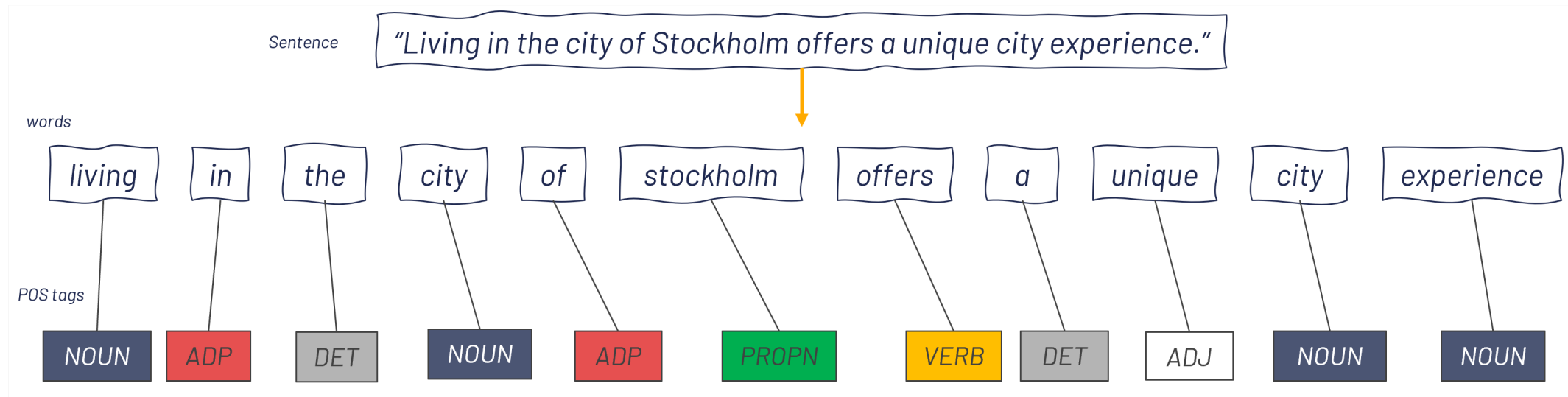




Part-of-Speech (POS) Tagging

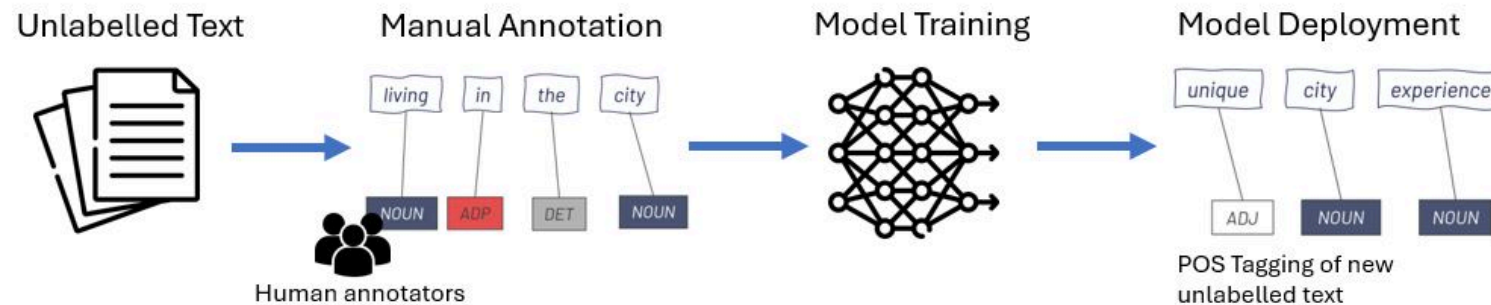
POS Tagging - What is it?

Part-of-Speech (POS) tagging is a process in natural language processing (NLP) where each word in a text (corpus) is assigned a tag that denotes its part of speech, such as NOUN, VERB, ADJECTIVE, etc., based on both its definition and its context.



POS Tagging - How does it work?

PoS tagging algorithms leverage **neural network architectures** to understand and predict the part-of-speech tags for each word in a given input sentence. These models are trained on large labeled datasets and can capture complex patterns and dependencies in the text.



One of the deep learning model to perform PoS tagging is [UDPipe](#).

POS Tagging - UDPipe Introduction

- UDPipe provides language-agnostic tokenization, tagging, lemmatization and dependency parsing of raw text, which is an essential part in natural language processing.
- The techniques used are explained in detail in the paper: "Tokenizing, POS Tagging, Lemmatizing and Parsing UD 2.0 with UDPipe", available at http://ufal.mff.cuni.cz/~straka/papers/2017-conll_udpipe.pdf.
- The package is available under the Mozilla Public License Version 2.0. Installation can be done as follows.

```
# Install packages (only once)
install.packages("udpipe")
```

UDPipe - Model Download

UDPipe provides **65** language models. When using this package, you need to download the chosen model and load it

```
# Import package
library(udpipe)
library(tidyverse)

# Download the English model to perform lemming (only once)
# udpipe_download_model(language = "english")

# Load the model before using it
ud_model <- udpipe_load_model("data/english-ewt-ud-2.5-191206.udpipe")
```

POS Tagging - Example

Let's extract the POS tags from a set of sentences.

```
# Annotate the sentences
annotated_text <- udpipe_annotate(ud_model, x = text_data$text, doc_id = text_data$Sentence_ID) |>
  as_tibble()

POS_tags <- annotated_text |>
  select(sentence, token, upos)

POS_tags
```

```
## # A tibble: 25 × 3
##   sentence                                token upos
##   <chr>                                <chr> <chr>
## 1 living in the city of Stockholm offers a unique ci... livi... VERB
## 2 living in the city of Stockholm offers a unique ci... in      ADP
## 3 living in the city of Stockholm offers a unique ci... the     DET
## 4 living in the city of Stockholm offers a unique ci... city    NOUN
## 5 living in the city of Stockholm offers a unique ci... of      ADP
## 6 living in the city of Stockholm offers a unique ci... Stoc... PROPN
## # i 19 more rows
```

udpipe_annotate

The function perform a series of operations on the sentences, creating a new dataframe where the **observations** are the **tokens** and containing the following information for each of them:

- the ID of the document, of the paragraph, of the sentences they come from
- the full sentence they come from
- the ID, the text, the lemma, the POS, and other information of the tokens

```
head(annotated_text, 3)
```

```
## # A tibble: 3 × 14
##   doc_id paragraph_id sentence_id sentence  token_id token lemma
##   <chr>      <int>      <int> <chr>      <chr>    <chr> <chr>
## 1 1          1          1 living in... 1      livi... live
## 2 1          1          1 living in... 2        in    in
## 3 1          1          1 living in... 3        the    the
## # i 7 more variables: upos <chr>, xpos <chr>, feats <chr>,
## #   head_token_id <chr>, dep_rel <chr>, deps <chr>, misc <chr>
```

udpipe_annotate



Therefore, the function perform the following operations:

- Splits the text into paragraphs
- Splits the paragraphs into sentences
- Tokenizes the sentences into single tokens
- Lemmatizes the tokens
- Assigns the Part of Speech to tokens
- Identify the dependencies of tokens

POS Tagging - Patent Data



Apply POS tagging on patent data to annotate patent abstracts.

```
# Import Patent Data
patent_data <- read_rds("data/patents.rds")

# Get the first 500 patents
patent_data <- patent_data |>
  head(500)

# Perform POS tagging on patent abstracts
patents_POS <- udpipe_annotate(ud_model, x = patent_data$abstract, doc_id = patent_data$id) |>
  as_tibble()
```

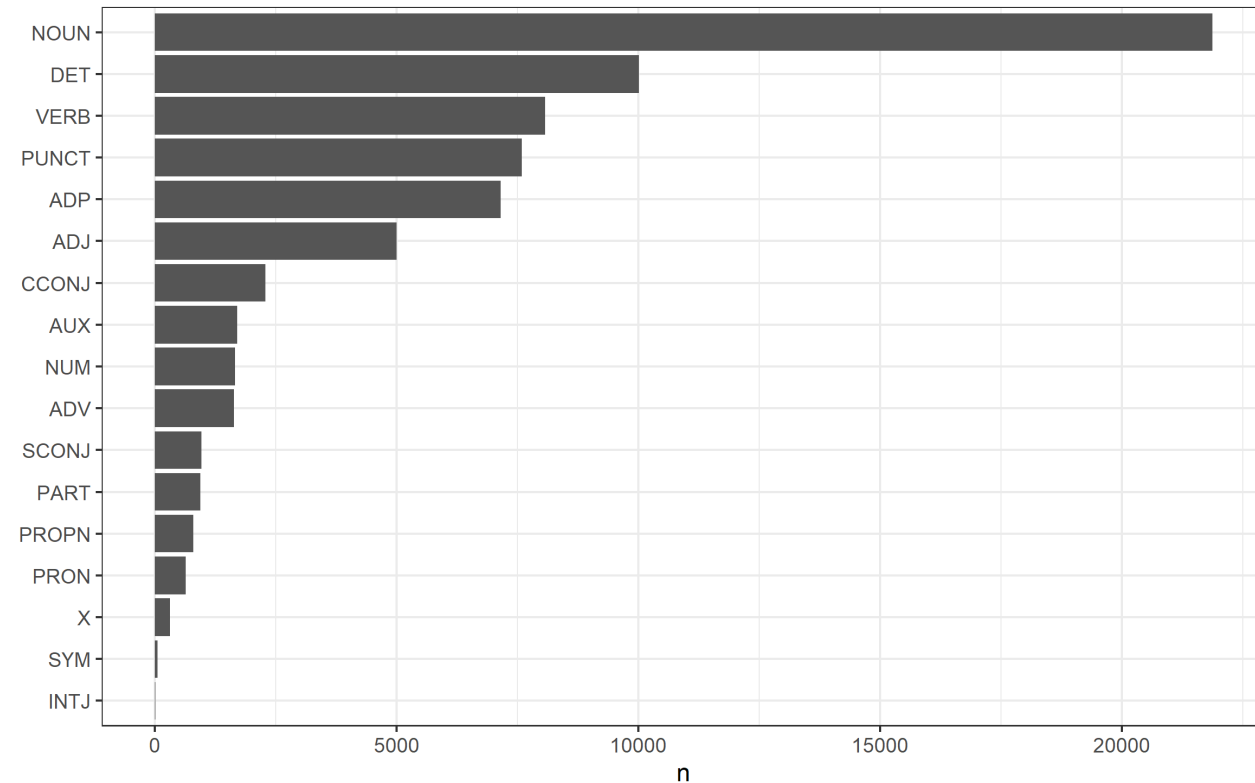
POS Tagging - Patent Data

In most languages, nouns (NOUN) are the most common types of words, next to verbs (VERB) and these are the most relevant for analytical purposes, next to the adjectives (ADJ) and proper nouns (PROPN).

```
# Plot frequency of POS tags
patents_POS |>
  count(upos) |>
  ggplot(aes(x = reorder(upos, n), y = n)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  xlab("") +
  theme_bw()
```

For a detailed list of all POS tags: visit <https://universaldependencies.org/u/pos/index.html>.

POS Tagging - Patent Data



POS Tagging - Patent Data

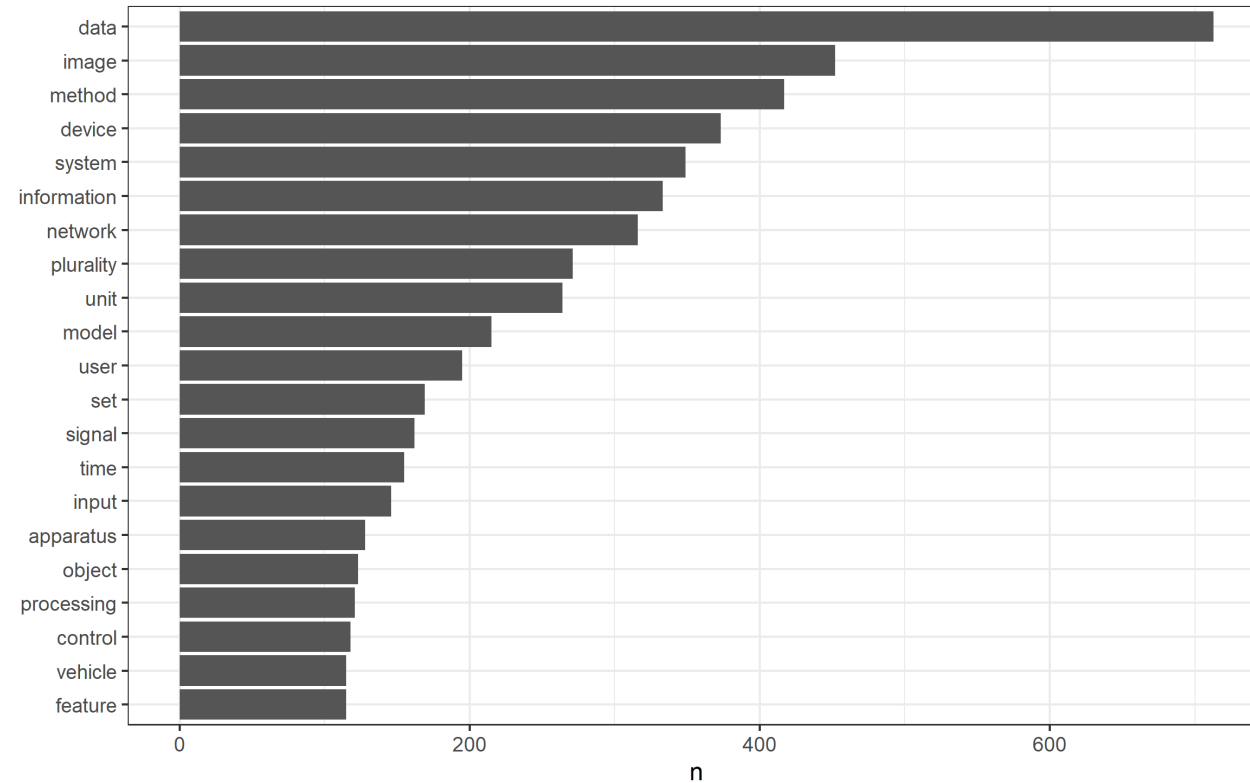
Parts of Speech tags are really interesting to extract easily the words you like to plot. You really don't need stopwords for doing this, just select nouns / verbs or adjectives and you have already the most relevant parts for basic frequency analysis.

```
# Plot the word frequency of the top 20 most common NOUNs in patent abstracts.
graph_noun <- patents_POS |>
  filter(upos == "NOUN") |>
  count(token) |>
  top_n(n= 20) |>
  ggplot(aes(x = reorder(token, n), y = n)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  xlab("") +
  theme_bw()
```

POS Tagging - Patent Data



graph_noun



POS Tagging - Patent Data

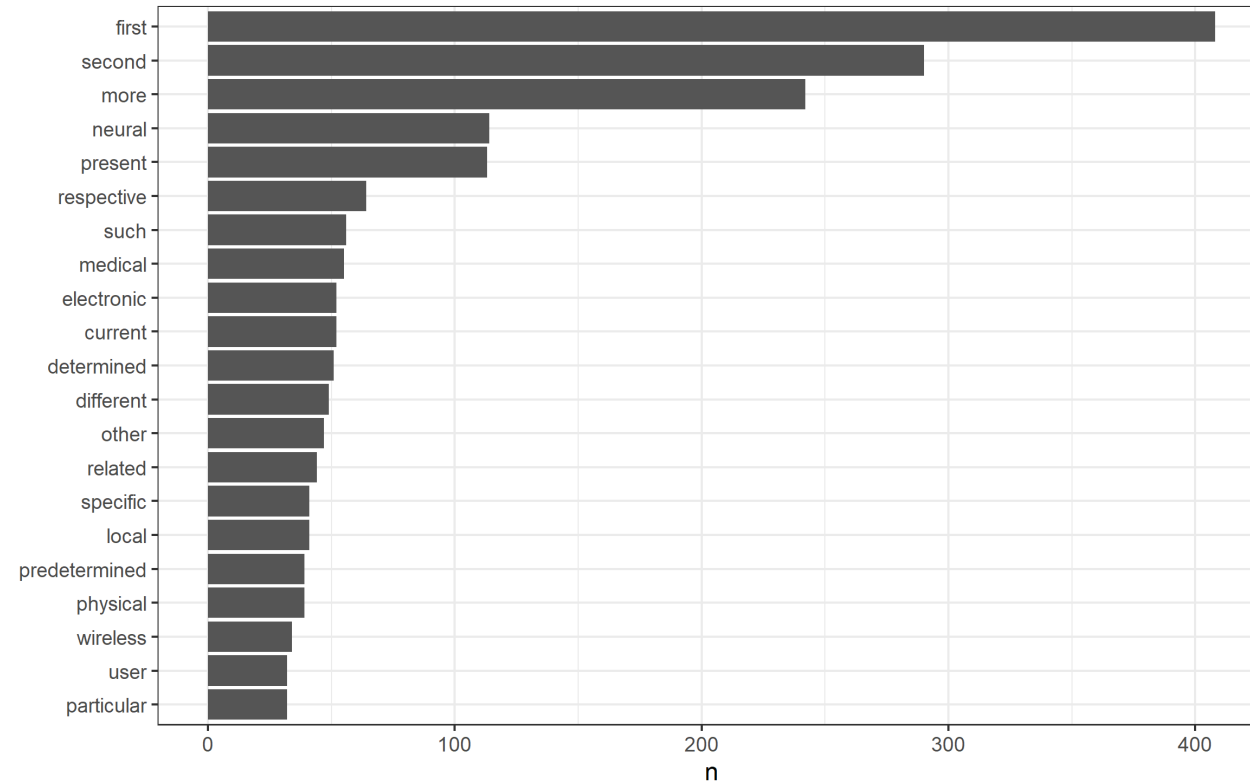


```
# Plot the word frequency of the top 20 most common ADJs in patent abstracts.
graph_adj <- patents_POS |>
  filter(upos == "ADJ") |>
  count(token) |>
  top_n(n= 20) |>
  ggplot(aes(x = reorder(token, n), y = n)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  xlab("") +
  theme_bw()
```

POS Tagging - Patent Data



graph_adj



POS Tagging - Patent Data

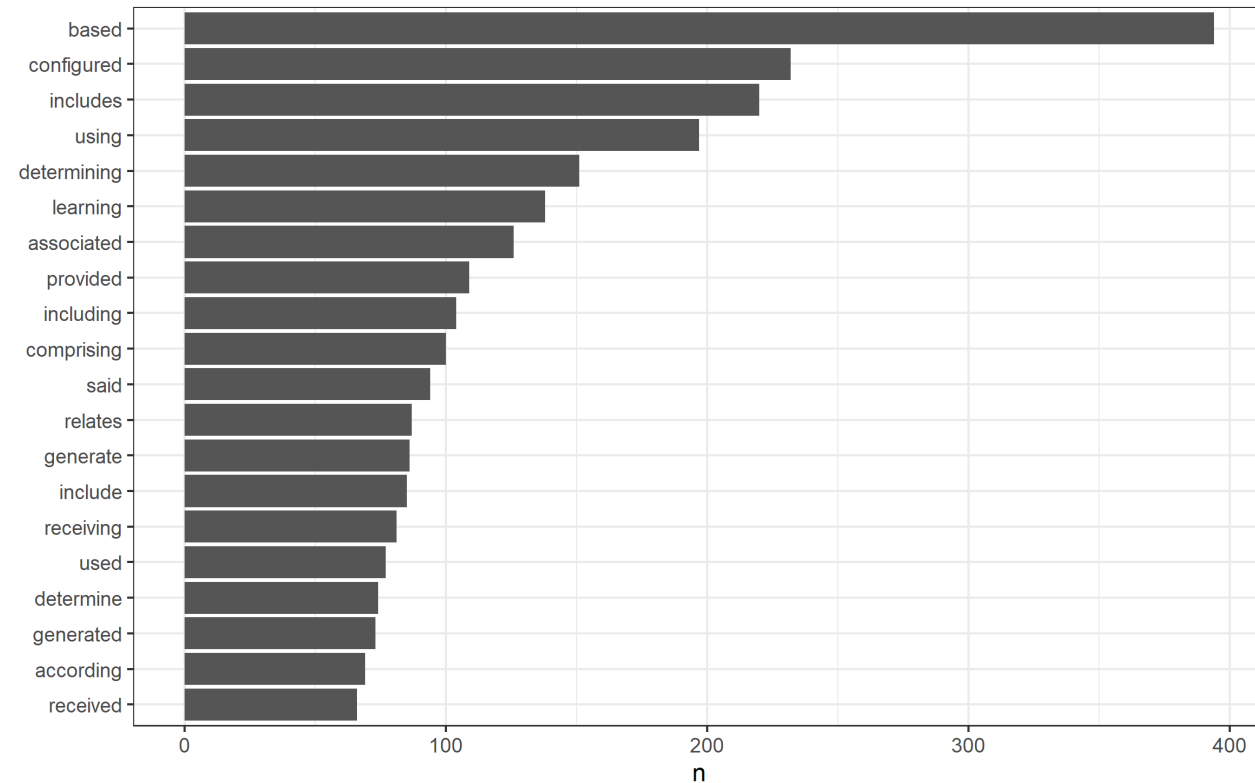


```
# Plot the word frequency of the top 20 most common VERBs in patent abstracts.
graph_vb <- patents_POS |>
  filter(upos == "VERB") |>
  count(token) |>
  top_n(n= 20) |>
  ggplot(aes(x = reorder(token, n), y = n)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  xlab("") +
  theme_bw()
```


POS Tagging - Patent Data



graph_vb

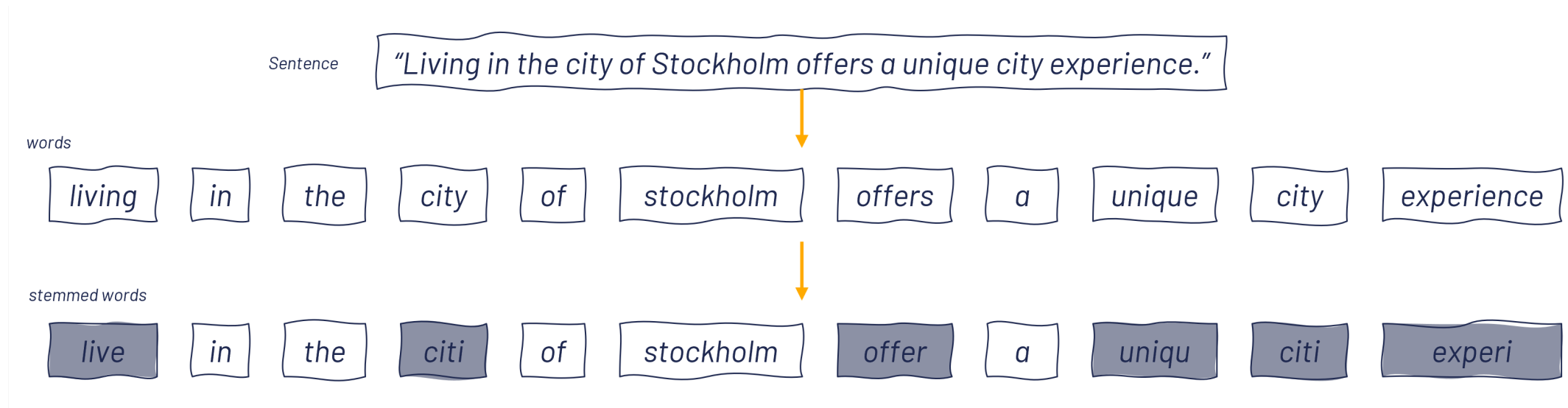




Stemming and Lemmatization

Stemming - What is it?

Stemming is the process of reducing inflected word to their **word root** form by chopping chops off prefixes or suffixes without considering the meaning of the word. Thus, it may **produce non-dictionary forms** (e.g., "city" to "citi", "better" to "bett", etc.). A popular stemmer is **Porter's stemming algorithm**



Stemming - Example



Perform Stemming over a set of sentences.

```
# Import packages
library(SnowballC)

# Tokenize the selected sentence and remove stop words
tokenized_abstracts <- tokenized_abstracts |>
  anti_join(stop_words)
```

Stemming - Example

Perform Stemming over a set of sentences.

```
# Stem the words of the selected sentence
stemmed_sentence <- tokenized_abstracts |>
  mutate(stem = wordStem(word))

stemmed_sentence
```

```
## # A tibble: 621,809 × 4
##   id          title                                word  stem
##   <chr>        <chr>                                <chr> <chr>
## 1 EP1712182_B1 Method of ultrasonic detection and lo... meth... meth...
## 2 EP1712182_B1 Method of ultrasonic detection and lo... ultr... ultr...
## 3 EP1712182_B1 Method of ultrasonic detection and lo... dete... dete...
## 4 EP1712182_B1 Method of ultrasonic detection and lo... loca... local
## 5 EP1712182_B1 Method of ultrasonic detection and lo... cont... cont...
## 6 EP1712182_B1 Method of ultrasonic detection and lo... agent agent
## # i 621,803 more rows
```

Stemming - Example



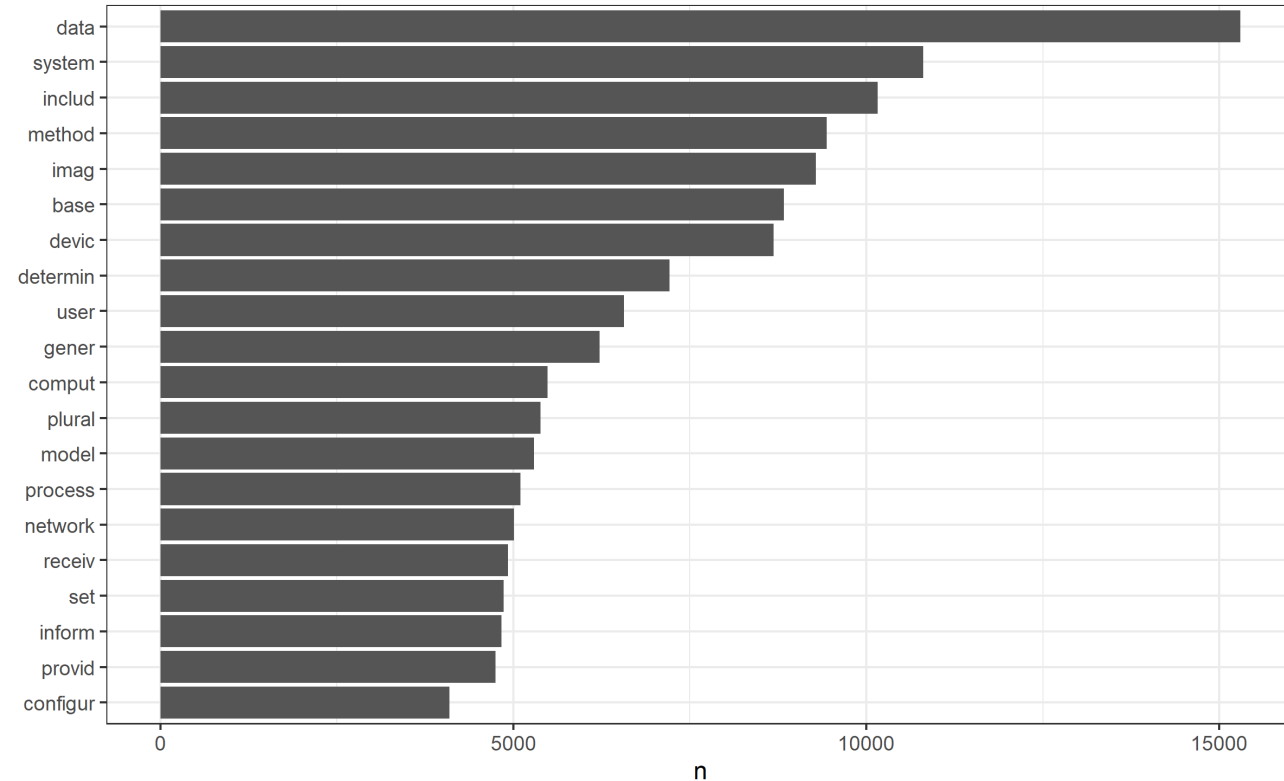
Let's visualize the 20 most frequent words after stemming.

```
top_20_word_freq <- stemmed_sentence |>
  count(stem, sort = TRUE) |>
  top_n(n = 20) |>
  # reorder levels of factor word wrt n
  mutate(stem = reorder(stem, n)) |>
  ggplot(aes(stem, n)) +
  geom_col() +
  xlab(NULL) +
  coord_flip() +
  theme_bw()
```

Stemming - Example

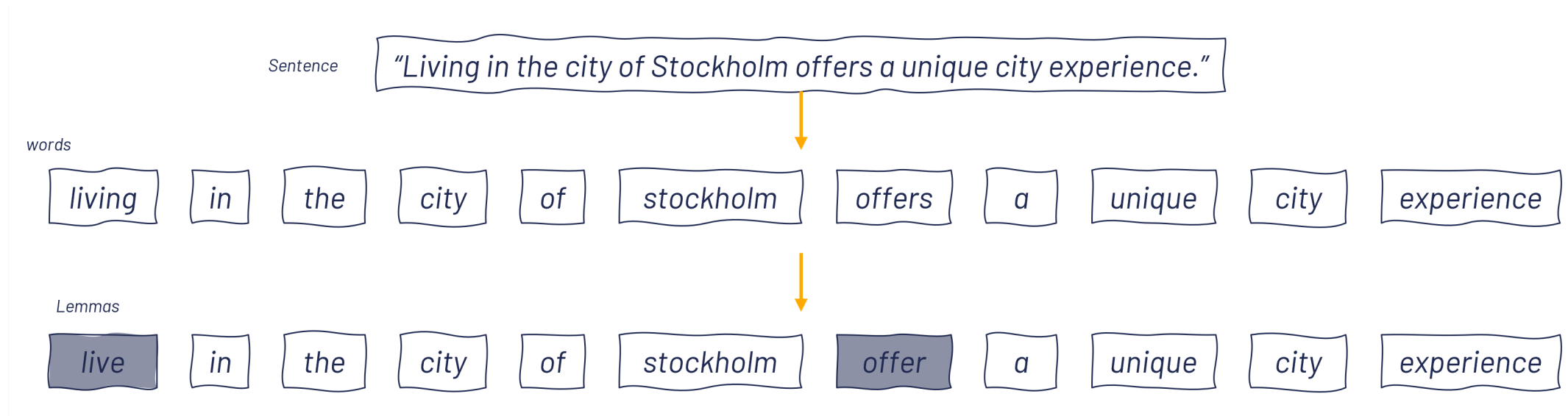


Let's visualize the 20 most frequent words after stemming.



Lemmatization - What is it?

Lemmatization is the process of reducing inflected words to their **dictionary form (lemma)** while still preserving their meaning.



Lemmatization - What is new?

- Produces **meaningful root words** that are actual words in the language (e.g., "running" to "run").
- Instead of using rules to cut words down to their roots (stems), lemmatization uses **algorithms trained on textual data** to perform morphological analysis of textual data. This will result in more accurate and reliable pre-processing for understanding the meaning of words.

We will implement it later with the **udpipe** package. An alternative is **spacyR**

Lemmatization - Example

Perform Lemmatization over a set of sentences.

```
# Import package
library(udpipe)

# Download the English model to perform lemming (only once)
#en <- udpipe_download_model(language = "english")

# Load the model before using it
ud_model <- udpipe_load_model("data/english-ewt-ud-2.5-191206.udpipe")

# Annotate sentences
patents_POS <- udpipe_annotate(ud_model, x = patent_data$abstract, doc_id = patent_data$id) |>
  as_tibble()
```

Lemmatization - Example

Perform Lemmatization over a set of sentences.

```
# Get lemmas of the words of the selected sentence
annotated_text <- patents_POS |>
  select(sentence, token, lemma, upos)|>
  anti_join(stop_words, by = c("lemma" = "word"))

annotated_text
```

```
## # A tibble: 41,837 × 4
##   sentence                                token lemma upos
##   <chr>                                <chr> <chr> <chr>
## 1 A method of ultrasonic detection and localiz... meth... meth... NOUN
## 2 A method of ultrasonic detection and localiz... ultr... ultr... ADJ
## 3 A method of ultrasonic detection and localiz... dete... dete... NOUN
## 4 A method of ultrasonic detection and localiz... loca... loca... NOUN
## 5 A method of ultrasonic detection and localiz... cont... cont... ADJ
## 6 A method of ultrasonic detection and localiz... agent agent NOUN
## # i 41,831 more rows
```

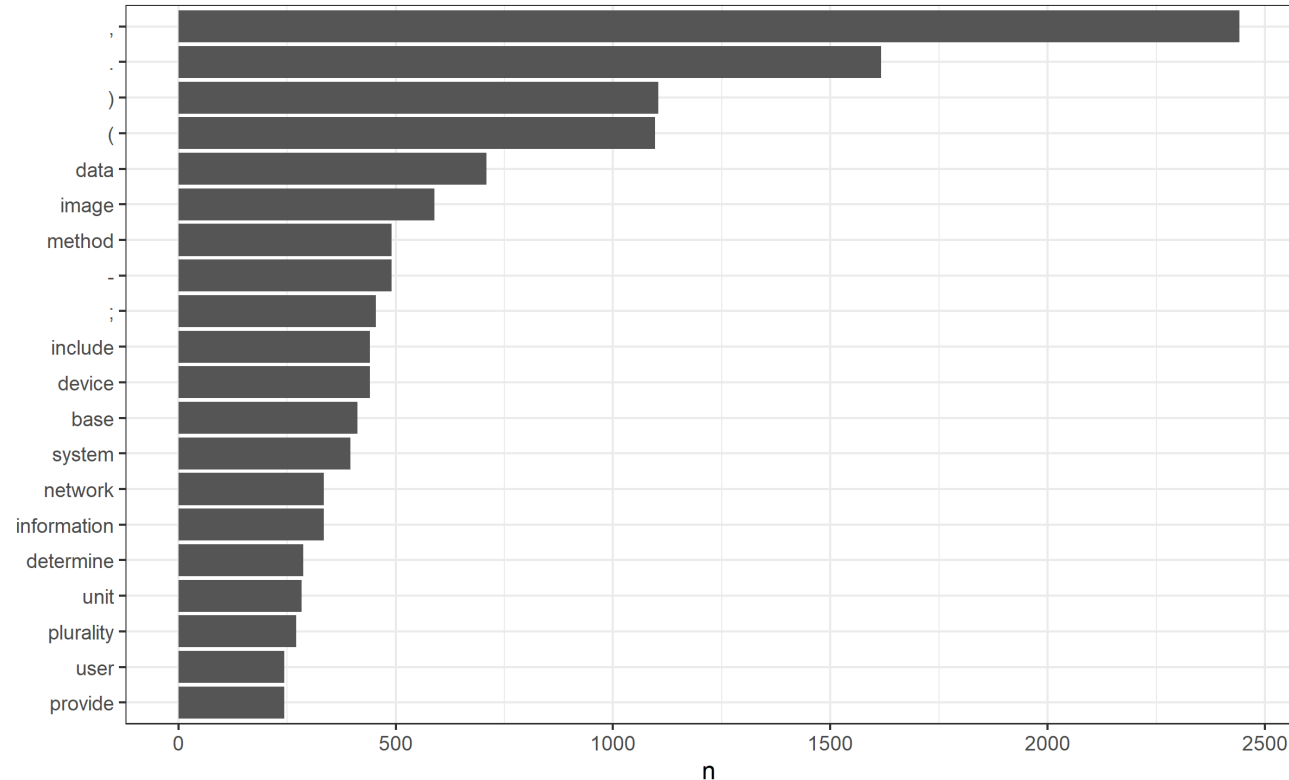
Lemmatization - Example

Let's visualize the 20 most frequent words after lemmatization

```
top_20_word_freq <- annotated_text |>
  count(lemma, sort = TRUE) |>
  top_n(n = 20) |>
  # reorder levels of factor word wrt n
  mutate(lemma = reorder(lemma, n)) |>
  ggplot(aes(lemma, n)) +
  geom_col() +
  xlab(NULL) +
  coord_flip() +
  theme_bw()
```

Lemmatization - Example

Let's visualize the 20 most frequent words after lemmatization



Since we did not use `unnest_tokens`, punctuation is still here (`udpipe_annotate` does not remove symbols).

Lemmatization - Example

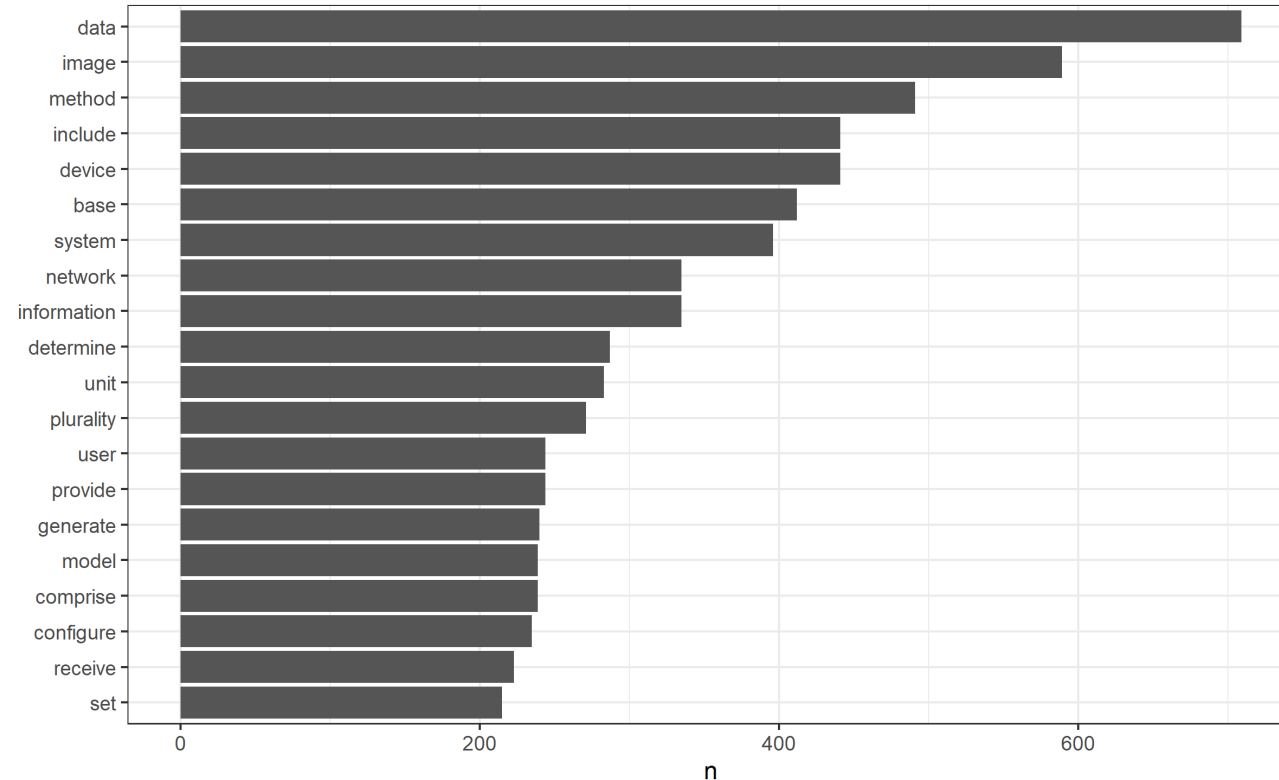


Let's do it again, deleting punctuation.

```
top_20_word_freq <- annotated_text |>
  filter(upos != "PUNCT") |>
  count(lemma, sort = TRUE) |>
  top_n(n = 20) |>
  # reorder levels of factor word wrt n
  mutate(lemma = reorder(lemma, n)) |>
  ggplot(aes(lemma, n)) +
  geom_col() +
  xlab(NULL) +
  coord_flip() +
  theme_bw()
```

Lemmatization - Example

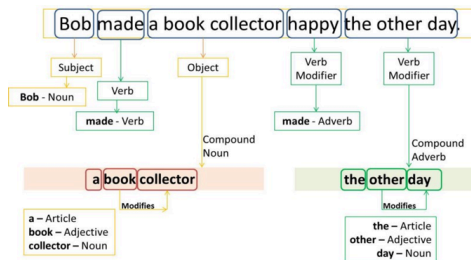
Let's visualize the 20 most frequent words after lemmatization and punctuation removal.



Intro to Textual Data - The Pipeline

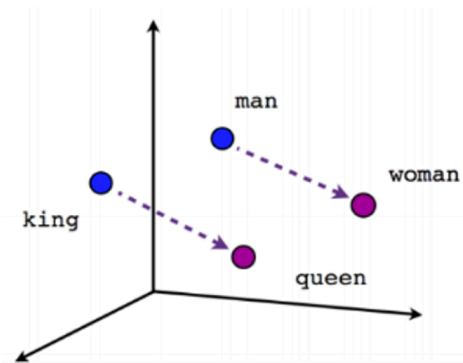
Text Preprocessing

This phase focuses on cleaning and standardizing the text to **reduce noise**. By removing irrelevant elements and harmonizing formats, preprocessing improves data quality and consistency. In addition, it helps manage the high dimensionality of textual data.



Text Representation

Text representation refers to the process of converting raw textual data into structured, **machine-readable formats** that can be analyzed computationally.



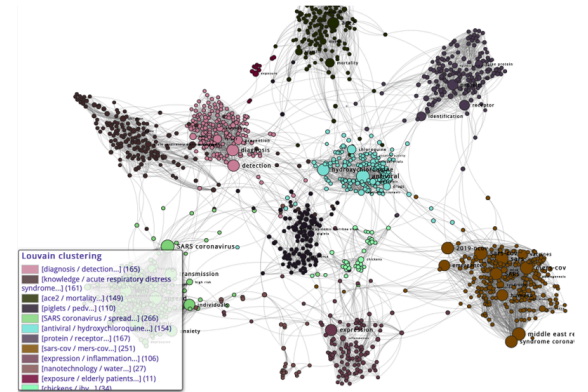
Text Modelling

Text modelling uses computational methods to extract information from text. It can be **supervised**, when models learn from labeled data to identify predefined categories or constructs, or **unsupervised**, when models explore the text to uncover patterns such as topics or clusters without prior labels.

tourists, the third ORDINAL most visited in the European Union ORG, and the most popular tourist destination. Its historic centre is listed by UNESCO ORG as a World Heritage ORG Site. [14] Host city for the Summer Olympics EVENT, Rome GPE is also the seat of several specialised agencies of the United ORG, such as the Food and Agriculture Organization ORG (FAO ORG), the World Food Programme ORG, and the International Fund for Agricultural Development ORG (IFAD ORG). The city also hosts the Secretariat of the Parliamentary Assembly ORG of the Union for the Mediterranean [15] (UfM ORG) as headquarters of many international business companies such as Eni ORG, Enel GPE, TIM ORG, S.p.A. ORG, and national and international banks such as Unicredit ORG and BNL ORG. Rome GPE business district is the home of many companies involved in the oil industry, the pharmaceutical industry, services. The presence of renowned international brands in the city have made Rome GPE an important center.

Statistical Analysis

Outputs of Text modelling are subsequently used as inputs for **downstream analyses**.





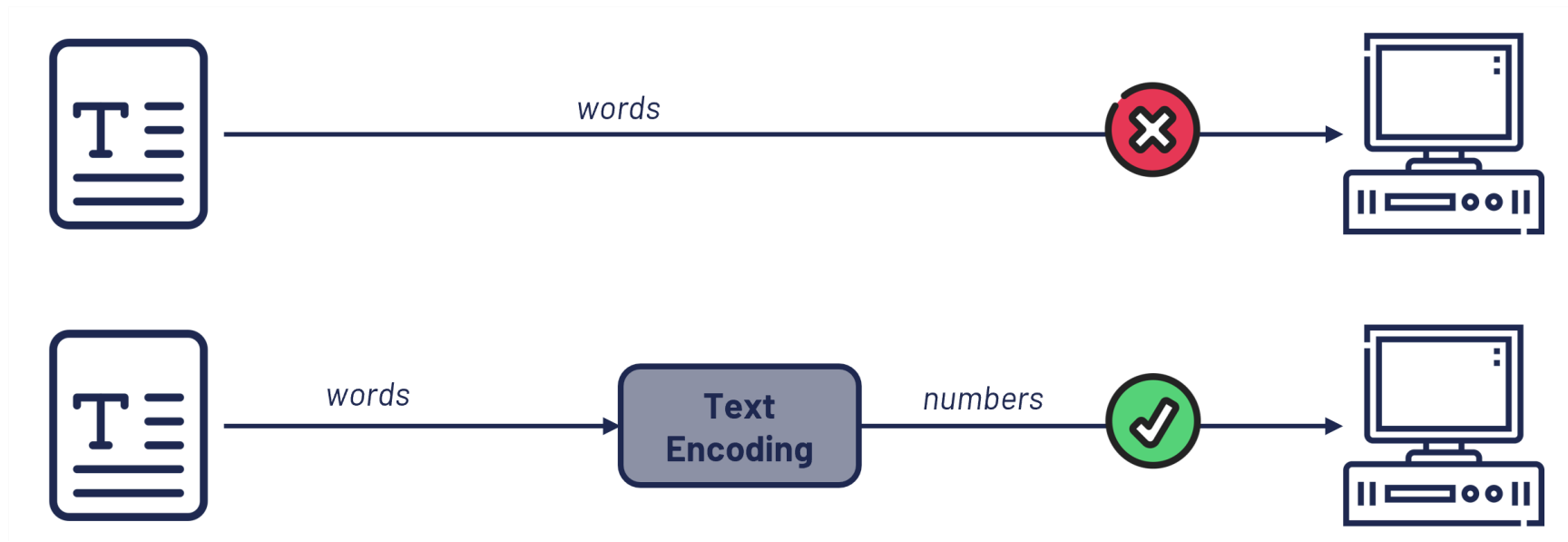
Text Encoding

Text Encoding - What is it?

Text encoding is a process to **convert text into machine-readable numbers** (numerical vectors).

We **cannot pass raw text into machines** as input until and unless we convert them into numbers.

Text Encoding **enables automated analysis of textual data**.

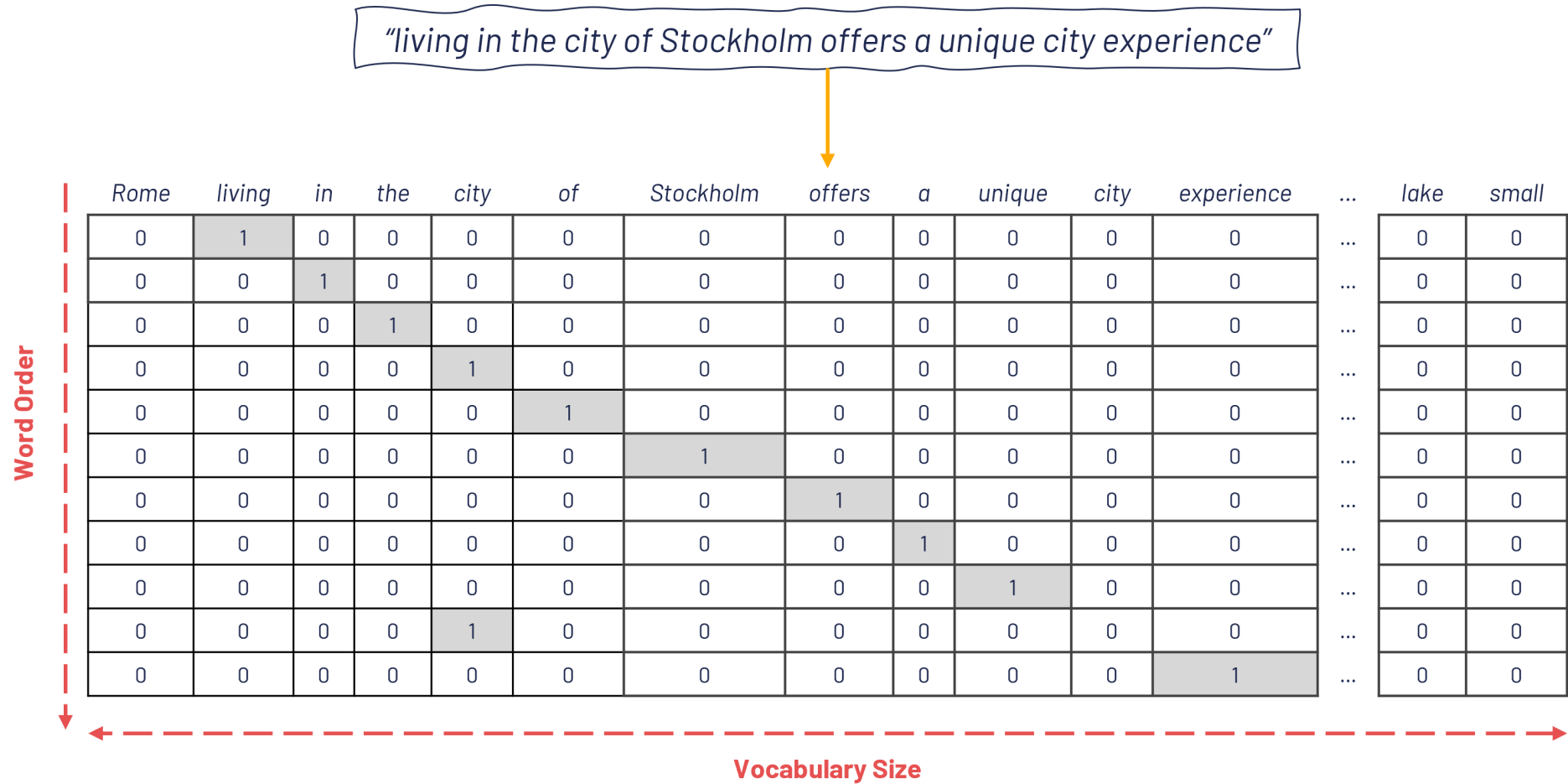


Text Encoding - How to perform it?

There are a lot of encodings techniques to convert text into numerical vectors, the most relevant ones are:

1. **One-hot encoding**: transform words into unique numerical representations;
2. **Bag-of-Words (BoW)**: captures word frequency disregarding order;
3. **TF-IDF encoding**: balances uniqueness and importance;
4. **Word Embeddings**: converts words into vectors, capturing semantic meaning.

One-hot encoding

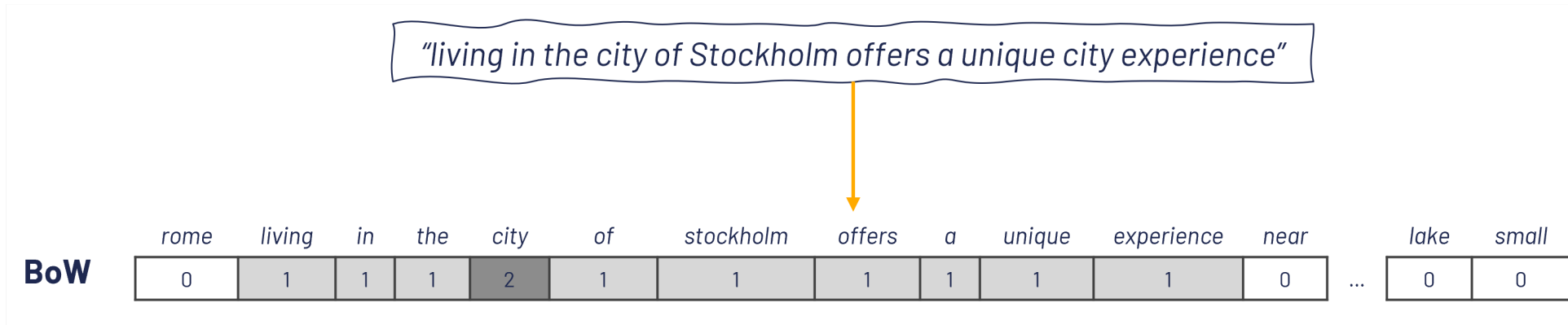


One-hot encoding

Observations:

- The **word order is preserved**
- **Polysemous words have same vector representation** (e.g., bank as “financial institution” and “side of a river”).
- **One row for each word**
- **One column for each word of the vocabulary**
- Each word is a **vector of size equal to vocabulary size** (word encoding)
- Each text is a **vertical stack of word encodings** (text encoding)
- **very large and very sparse representation** of text.

Bag-of-Words (BoW) - What is it?



Observations:

- **One row for each text**
- **One column for each word of the vocabulary**
- Treating each text as an **unordered collection of words**
- **Focuses on frequency**, not order
- **very large and very sparse representation** of document corpus

Bag-of-Words (BoW) - for Document-Term Matrix

To represent an entire corpus of documents with BOW, I can use a representation called **Document Term Matrix (DTM)**.

	text	mining	is	to	find	useful	information	from	text	mined	dark	came
D1	1	1	1	1	1	1	1	1	1	0	0	0
D2	0	0	1	0	0	1	1	1	1	1	0	0
D3	0	0	0	0	0	0	0	0	0	0	1	1

DTM is one of the most common structures that text mining packages work with. This is a matrix where:

- **each row** represents one **document** (such as a book or article or sentence),
- **each column** represents one **term**, and
- **each value** (typically) contains the **number of appearances** of that term in that document (as in BOW).

Document-Term Matrix - Patent Data



Let's convert the patent titles to a Document Term Matrix (dtm).

```
# Import packages
library(tidyverse)
library(tidytext)

# Import patent dataset
patent_data <- read_rds("data/patents.rds")

# Tokenize patent titles
tokenized_title <- patent_data |>
  unnest_tokens(output = word, input = title)

# Create the dtm of patent titles
dtm <- tokenized_title |>
  count(id, word)
```


Document-Term Matrix - Patent Data

Let's convert the patent titles to a Document Term Matrix (dtm).

```
# Create the dtm of patent titles
dtm <- dtm |>
  cast_dtm(id, word, n)

dtm
```

```
## <<DocumentTermMatrix (documents: 9818, terms: 6981)>>
## Non-/sparse entries: 84515/68454943
## Sparsity           : 100%
## Maximal term length: 22
## Weighting           : term frequency (tf)
```

Document-Term Matrix - Patent Data

Let's **visualize** the Document Term Matrix (dtm) of patent titles.

```
dtm
```

```
## # A tibble: 9,818 × 6,944
##   id          a      à abandoned abbreviation abdominal aberrant
##   <chr>      <dbl> <dbl>      <dbl>          <dbl>        <dbl>    <dbl>
## 1 EP171218...    0    0          0              0            0        0
## 2 EP194130...    0    0          0              0            0        0
## 3 EP212656...    0    0          0              0            0        0
## 4 EP216954...    0    0          0              0            0        0
## 5 EP224087...    0    0          0              0            0        0
## 6 EP225288...    0    0          0              0            0        0
## # i 9,812 more rows
## # i 6,937 more variables: aberration <dbl>, aberrations <dbl>,
## #   abilities <dbl>, abnormal <dbl>, abnormalities <dbl>,
## #   abnormality <dbl>, about <dbl>, absence <dbl>,
## #   absorbent <dbl>, absorption <dbl>, abstract <dbl>,
## #   abstraction <dbl>, abundance <dbl>, abuse <dbl>,
## #   abusive <dbl>, ac <dbl>, accelerate <dbl>, ...
```

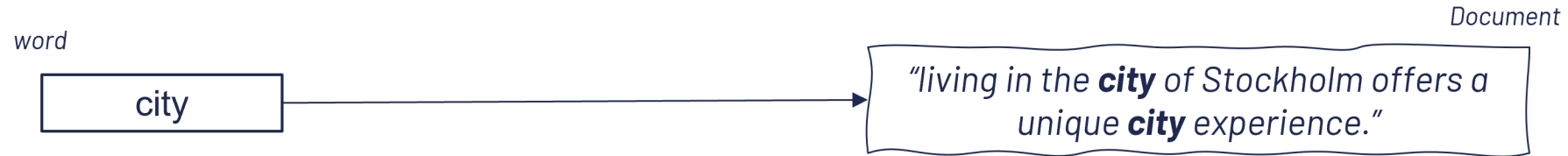
TF-IDF Encoding - TF-IDF Metric

The TF-IDF encoding uses the **TF-IDF metric** to encode words into vectors. TF-IDF metric is composed of two terms:

1. **Term Frequency (TF)**
2. **Inverse document frequency (IDF)**

TF-IDF Metric - TF term

Term Frequency (TF) measures how frequently a word occurs in a document (or sentence).



In the example the word "city" appears 2 times in a document that has 11 words. Therefore, the TF is $2/11 = 0.182$

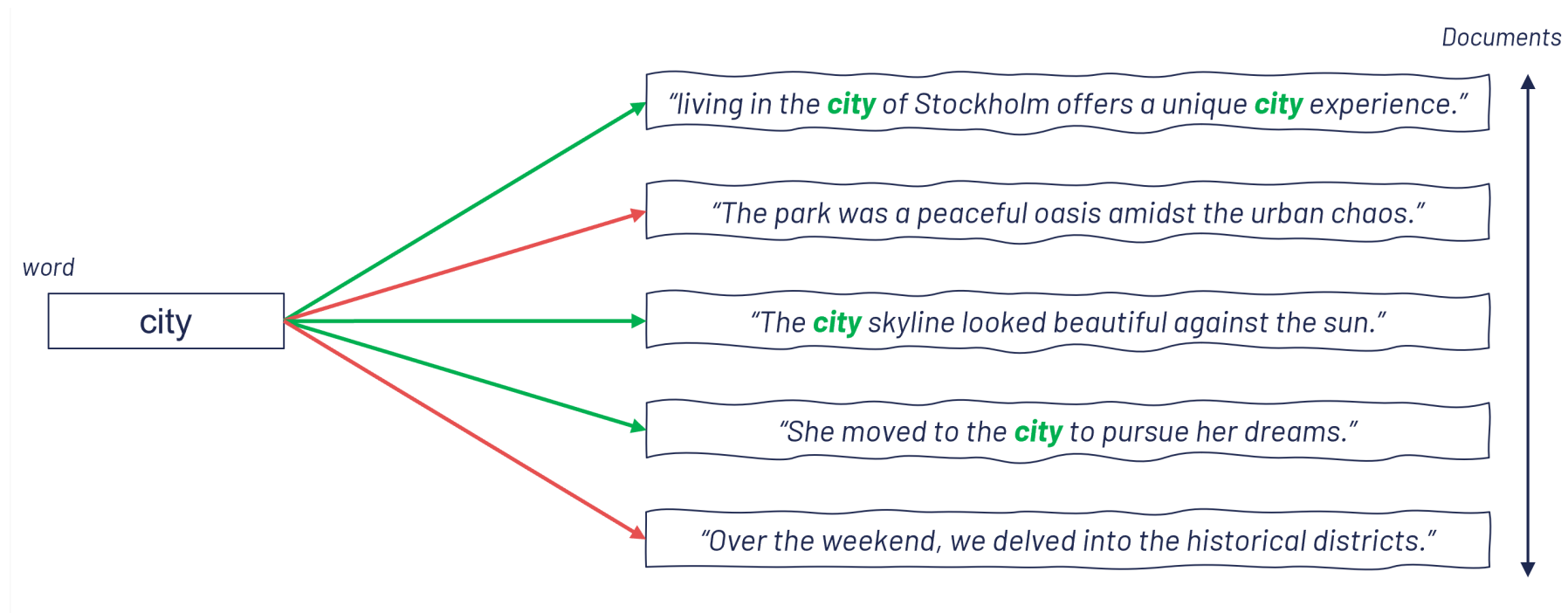
TF-IDF Metric - IDF term

Inverse document frequency (IDF) measures how rare a term is across multiple documents. It is calculated as follows:

$$idf(\text{term}) = \ln \left(\frac{n_{\text{documents}}}{n_{\text{documents containing term}}} \right)$$

TF-IDF Metric - IDF term

Inverse document frequency (IDF) measures how rare a term is across multiple documents.

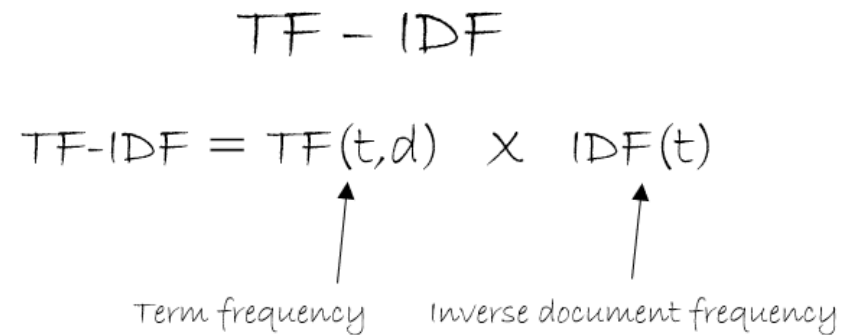


In the example, we have a collection of 5 documents and the word "city" appears in 3 of those documents. Therefore, the IDF for "city" is **$\text{Log}_e(5/3) = 0.511$**

TF-IDF Metric

TF-IDF combines both metrics to give a score that highlights the importance of a term in a document relative to a collection of documents (corpus).

$$\text{TF-IDF} = \text{TF}(t,d) \times \text{IDF}(t)$$


Term frequency Inverse document frequency

Using the previous example, if "city" has a TF of 0.182 in the first document and an IDF of 0.511, the TF-IDF score for "city" in that document would be **0.182 x 0.511 = 0.092**

Notice: depending on the number of times a word occurs in a documents the TF value can change, whereas the IDF value remains constant, until and unless new documents are getting added to the data corpus.

TF-IDF Metric - Patent Data



Let's calculate the TF-IDF of words for each patent abstract.

```
# Import packages
library(tidyverse)

# Import patent dataset
patent_data <- read_rds("data/patents.rds")
```


TF-IDF Metric - Patent Data



Let's calculate the TF-IDF of words for each patent abstract.

```
# Calculate the occurrences of each word for each patent
patent_words <- patent_data |>
  unnest_tokens(word, abstract) |>
  count(id, word, sort = TRUE)
```

TF-IDF Metric - Patent Data

Let's calculate the TF-IDF of words for each patent abstract.

```
patent_words
```

```
## # A tibble: 535,312 × 3
##   id          word      n
##   <chr>      <chr> <int>
## 1 EP3614349_A1 sub      64
## 2 US2020250731_A1 the      64
## 3 US2020195514_A1 the      43
## 4 US2020376295_A1 the      43
## 5 EP3770583_A1 the      40
## 6 US2020250731_A1 or       39
## # i 535,306 more rows
```

TF-IDF Metric - Patent Data

Let's calculate the TF-IDF of words for each patent abstract.

Notice that IDF and thus TF-IDF are zero for extremely common words.

```
# Compute the TF-IDF score
patent_words <- patent_words |>
  bind_tf_idf(word, id, n)
```

```
patent_words
```

```
## # A tibble: 535,312 × 6
##   id          word      n      tf      idf    tf_idf
##   <chr>      <chr> <int>  <dbl>  <dbl>    <dbl>
## 1 EP3614349_A1  sub      64 0.154  4.35    0.669
## 2 US2020250731_A1 the      64 0.108  0.0199 0.00215
## 3 US2020195514_A1 the      43 0.121  0.0199 0.00241
## 4 US2020376295_A1 the      43 0.117  0.0199 0.00231
## 5 EP3770583_A1  the      40 0.125  0.0199 0.00248
## 6 US2020250731_A1 or       39 0.0659 0.715   0.0471
## # i 535,306 more rows
```

TF-IDF Encoding

Let's exploit the TF-IDF metric to explain the TF-IDF encoding

"living in the city of Stockholm offers a unique city experience"



	Rome	living	in	the	city	of	Stockholm	offers	a	unique	experience	near	...	lake	small
TF-IDF	0.000	0.299	0.000	0.000	0.1333	0.000	0.324	0.124	0.000	0.05	0.103	0.123		0.000	0.000

Observations:

- **One row for each text**
- **One column for each word of the vocabulary**
- Treating each text as an **unordered collection of words**
- **Rare words have a higher score**
- **Common ones have a lower score**
- **very large and very sparse representation** of document corpus

TF-IDF Metric



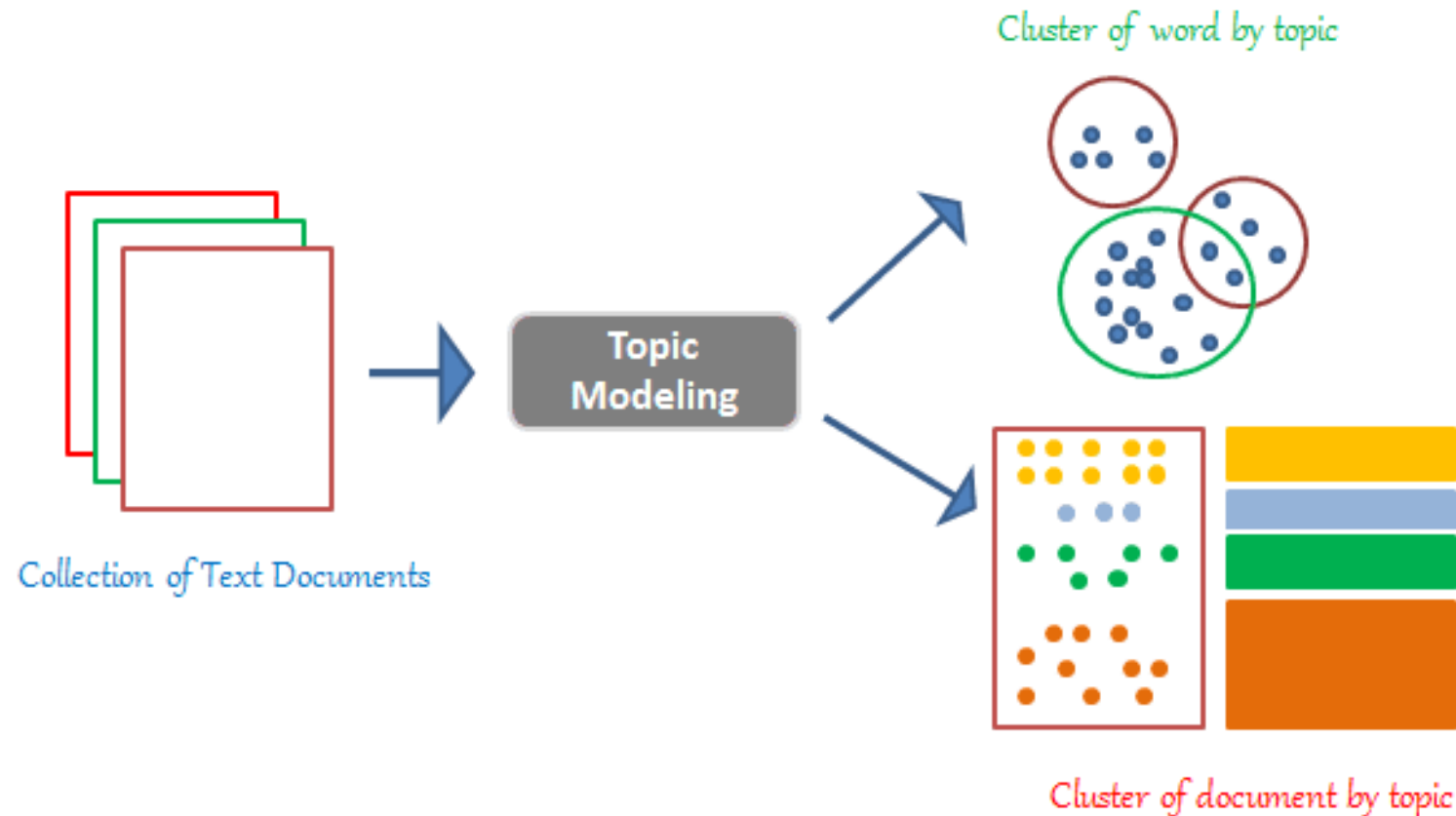
Let's create the Document Term Matrix of patent titles using TF-IDF score.

```
patent_words |>  
  cast_dtm(id, word, tf_idf)
```

```
## <<DocumentTermMatrix (documents: 9818, terms: 17979)>>  
## Non-/sparse entries: 535312/175982510  
## Sparsity           : 100%  
## Maximal term length: NA  
## Weighting          : term frequency (tf)
```

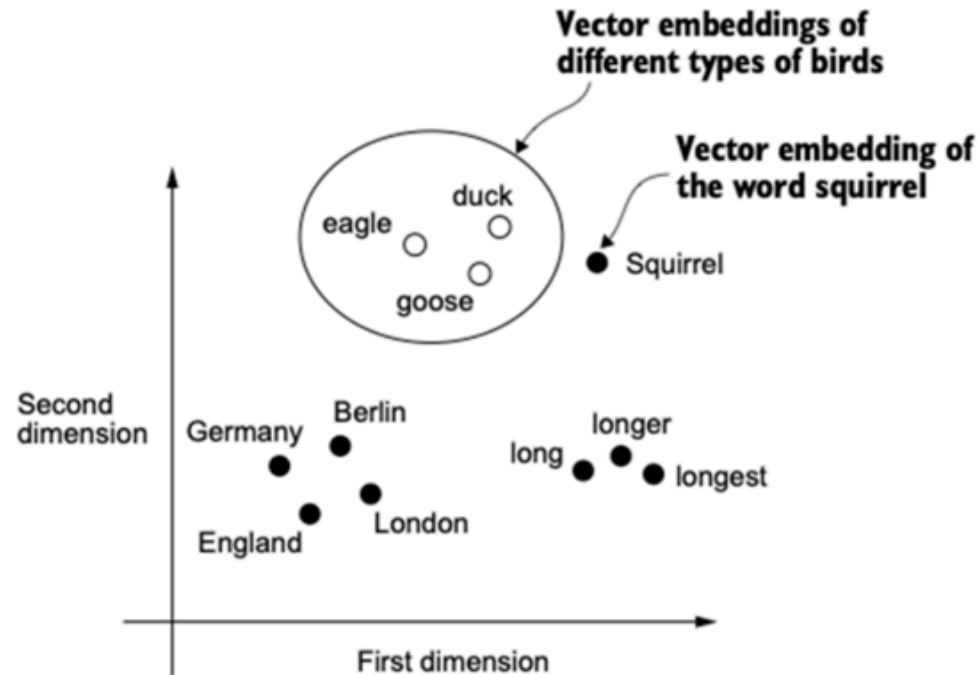
Topic Modelling - What is it?

Topic modeling is a statistical method used in natural language processing to discover **abstract topics** within a collection of documents. It helps in uncovering hidden thematic structures in a large corpus of text, making the information more manageable and interpretable.



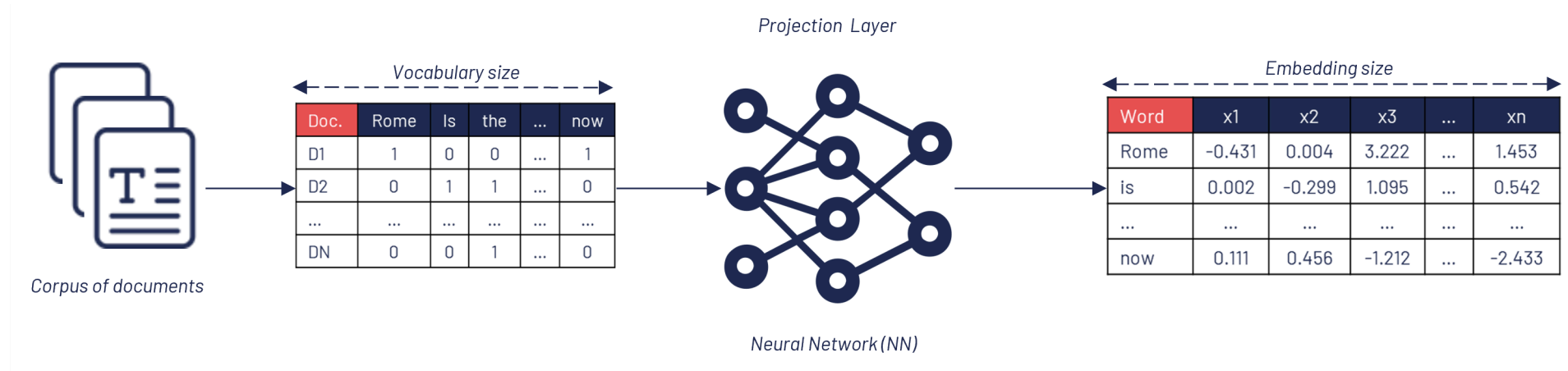
Word Embeddings - What are they?

Word embeddings aims at **mapping semantic meaning into a vector space**. Each word in a dictionary is represented by a numeric vector, where the **distance between vectors reflects the difference in meanings between the words**. Similar words (i.e., having a similar meaning) have similar embeddings.



Word Embeddings - What are they?

Word embeddings are created using Neural Networks.



These Neural Network trained to learn relationships between words constitute **Language Models**.

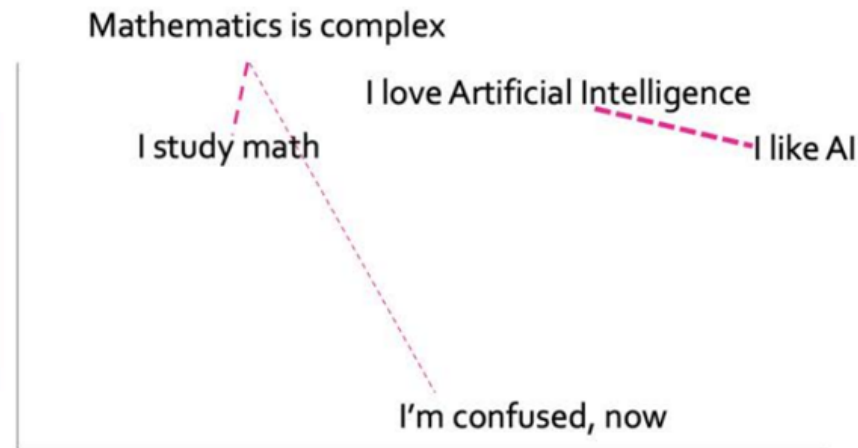
Word Embeddings - What are they?

The advantages of creating word embeddings using Neural Networks are:

- **Dimensionality Reduction**: previous encoding techniques requires a vector with a length equal to the size of the vocabulary, leading to very high-dimensional and sparse vectors.
- **Capturing Semantic Relationships**: with previous encoding techniques each words is represented as an independent and orthogonal vector, failing to capture any semantic relationships between words.

Word Embeddings - for Text Similarity

There are also embeddings for sentences, paragraphs, or whole documents. In this lecture, we can consider **sentence embeddings** as the average of the word embeddings for each word in a sentence.



Word embeddings can then be used in various NLP tasks, including **text similarity** tasks where the goal is to determine how similar two pieces of text are.

Cosine Similarity - for Text Similarity

To estimate the similarity of two texts, based on their vector representations, we can use **Cosine similarity**, a measure that calculates the cosine of the angle between two non-zero vectors in a multi-dimensional space.

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

For example, word embeddings for *king* and *queen* have a high cosine similarity due to their related meanings.

Word Embeddings - with Word2Vec

The earlier and most popular word embedding algorithm is **Word2Vec**.

The idea behind word embeddings is that **words that occur in similar contexts tend to be closer to each other in vector space**.

A neural network learns to associate a vector representation of a target word that is beneficial for predicting the target word based on the context provided by the surrounding words.

Word2Vec - Patent Data



Create word embeddings using Word2Vec.

Reference: <https://www.geeksforgeeks.org/word2vec-using-r/>

```
# Install necessary packages (only once)
#install.packages(c("word2vec", "data.table", "tm"))

# Load the libraries
library(data.table)
library(word2vec)
library(tidyverse)

# load patent data and drop missing values
patent_data <- read_rds("data/patents.rds") |>
  drop_na()

# Fit Word2Vec algorithm on patent abstracts to get word embeddings
word2vec_model = word2vec(x = patent_data$abstract, type = "cbow", dim = 15, iter = 20)
```

Word2Vec - Patent Data

Now, let's visualize the 15-dimensional vectors generated by Word2Vec for the word "model".

```
# checking embeddings
word2vec_embeddings <- predict(word2vec_model, "model", type = "embedding") |>
  as_tibble()

word2vec_embeddings
```

```
## # A tibble: 1 × 15
##       V1      V2      V3      V4      V5      V6      V7      V8      V9      V10
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 -0.589 -1.19 -1.08  1.80 -0.767 0.589 0.897 -1.60 -0.256 -0.351
## # i 5 more variables: V11 <dbl>, V12 <dbl>, V13 <dbl>,
## #   V14 <dbl>, V15 <dbl>
```

Word2Vec - Patent Data

Now, let's compute the distance between word embeddings to find the top five vectors which are more similar to the word "model".

```
# finding lookalike vecotors  
embeddings_lookslake <- predict(object = word2vec_model, "model", type = "nearest", top_n = 5)
```

Word2Vec - Patent Data

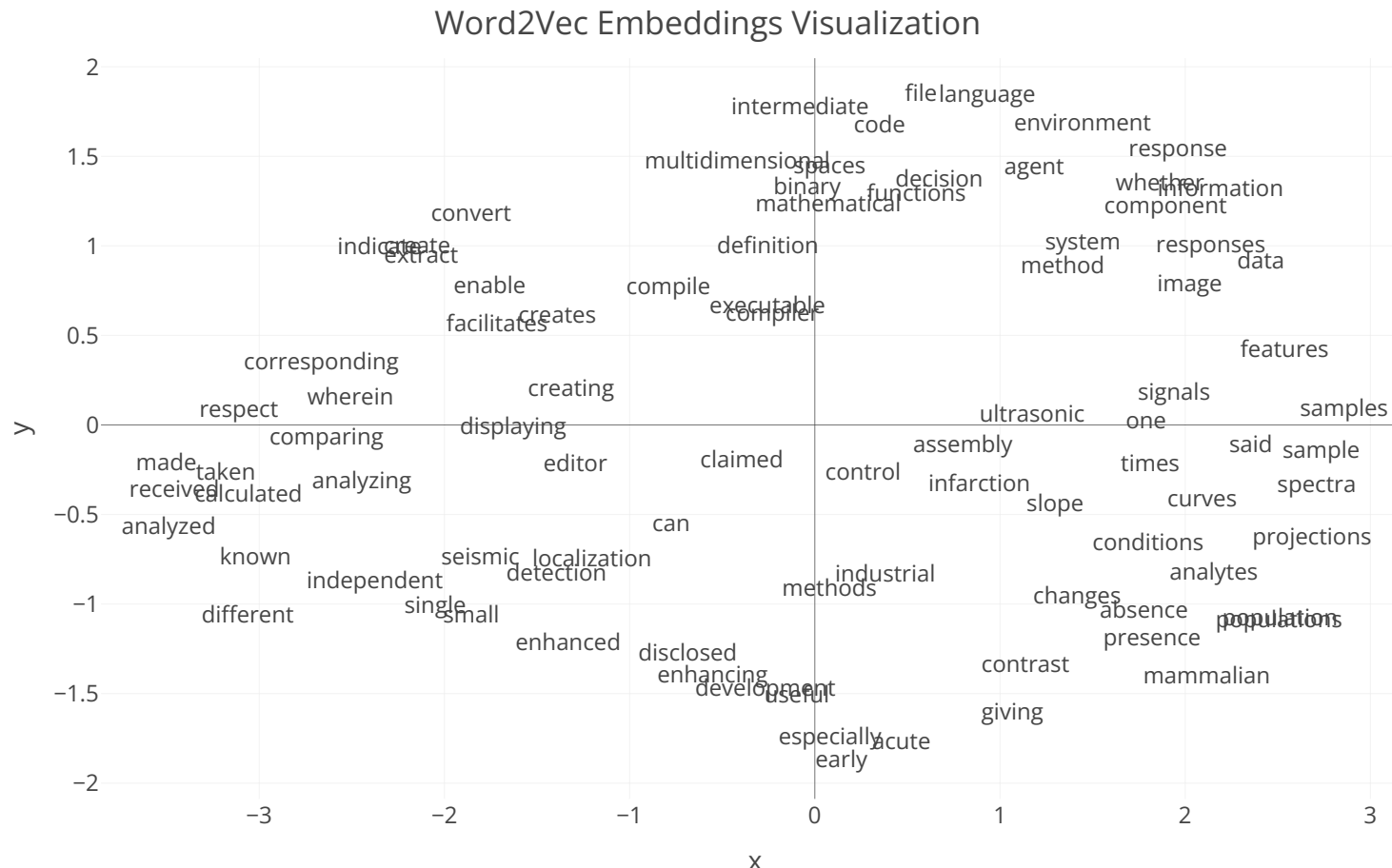
Visualize the top five vectors which are more similar to (closest to) the word "model" in the vector space

```
embeddings_lookslike$model
```

##	term1	term2	similarity	rank
## 1	model	algorithm	0.9708796	1
## 2	model	classifier	0.9163882	2
## 3	model	models	0.9077312	3
## 4	model	machine	0.8984943	4
## 5	model	regression	0.8933875	5

Word2Vec - Patent Data

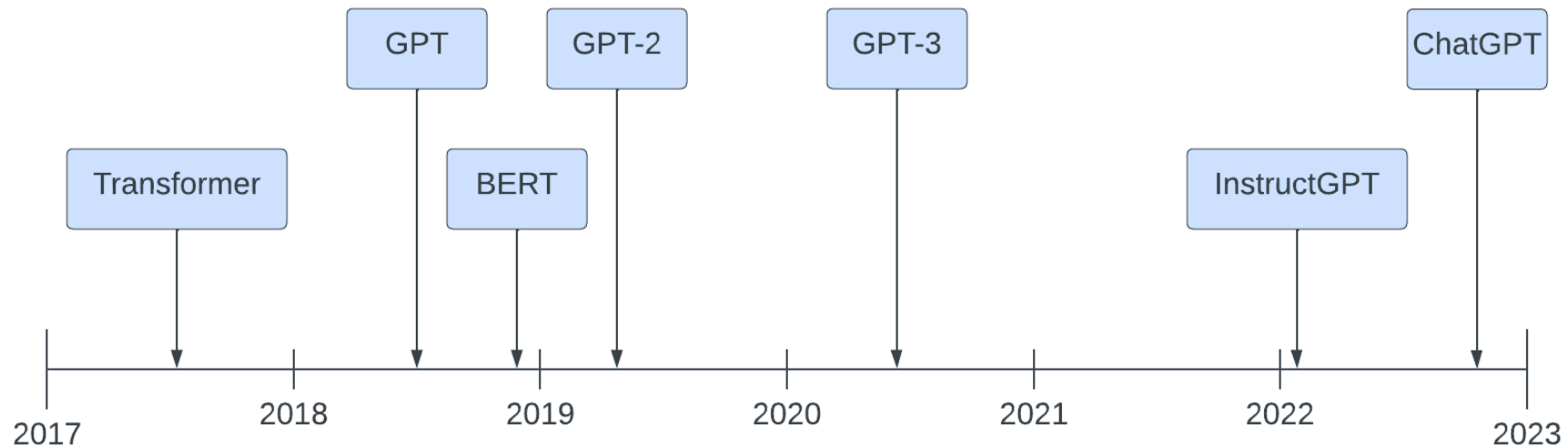
Now, let's visualize the embedding space created by Word2Vec for patent abstracts. This plot requires three packages, which should be installed and loaded: "ggrepel", "plotly", and "umap".



Word Embeddings - LLMs Introduction

In 2017, researchers at Google published a [paper](#) that proposed a novel neural network architecture for sequence modeling, called **Transformer**.

Large language models (LLMs) are machine learning algorithms based on transformer architecture which are **trained on massive amounts of text data**, sometimes comprising billions of words.



Word Embeddings - LLMs Introduction

These models are labeled as "large" because they contain **billions of trained parameters**. LLMs use word embeddings as internal representations of textual data. The most famous LLMs are:

- **BERT** (Bidirectional Encoder Representations from Transformers)
- **ChatGPT** (Generative Pre-trained Transformer)

After training, these models can be **fine-tuned** on domain-specific data. The fine tuning process enables to tailor the word embeddings to the specific task at hand, resulting in a precise representation of meaning.

Word Embeddings - What is new?

LLMs are refined during the training process. The benefit of optimizing these embeddings within the LLM training, rather than using pre-trained embeddings like Word2Vec, is that they become **contextual word embeddings**.

Previous encoding techniques do not provide any mechanism to handle synonyms or the multiple meanings (polysemy) of words. Words with the same spelling will have identical embeddings. With contextual embeddings, however, the meaning of a word is dynamically **adjusted based on the surrounding context**. This allows the model to differentiate between various senses of a word and accurately represent synonyms in context.

Word Embeddings - Contextual Embeddings

Using contextual word embeddings the same word can have **different embeddings depending on its context**.

For example, consider what comes to mind when you see the word **“flies”**. You might think of annoying insects, but if you were given more context, like “time flies like an arrow”, then you would realize that “flies” refers to the verb instead.

Now, consider the following sentences:

1. "time flies like an arrow"
2. "fruit flies like a banana"

Word Embeddings - Contextual Embeddings

We can create a representation for “flies” that incorporates this context by combining all the word embeddings in different proportions, perhaps by assigning a larger weight to the word embedding for “time” and “arrow”.

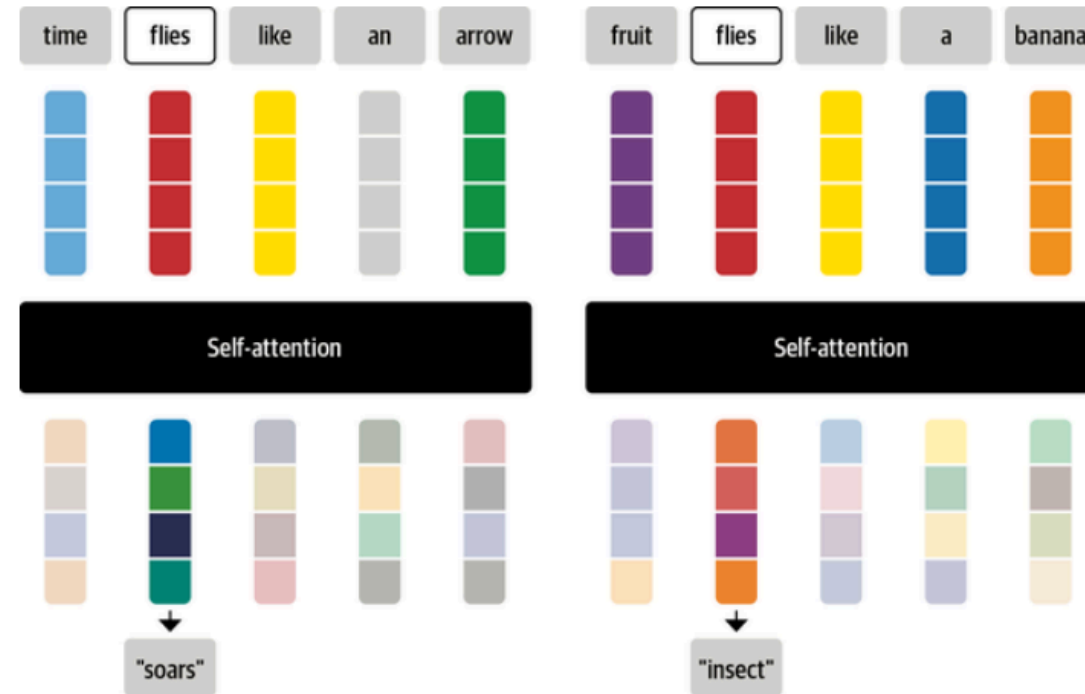


Figure 3-3. Diagram showing how self-attention updates raw token embeddings (upper) into contextualized embeddings (lower) to create representations that incorporate information from the whole sequence

Visualization from: <https://www.oreilly.com/library/view/natural-language-processing/9781098136789/>

Word Embeddings - with BERT

We can employ BERT to get the word embeddings of an input text. To do it, you need a Python environment that can be called from R through the package **reticulate**.

As Python environment, we suggest to install Anaconda3.

<https://docs.anaconda.com/anaconda/install/>

Once Anaconda is installed, you should also install the libraries **transformers** and **torch** through the Anaconda Prompt:

```
pip install pytorch  
pip install transformers
```

Then, install reticulate in RStudio

```
#install.packages("reticulate")
```

Word Embeddings - with BERT

```
library(reticulate)

# Use the desired Python environment (adjust the path to your specific environment)
use_python("C:/Users/Vito Giordano/anaconda3")

# Import transformers library
transformers <- import("transformers")
torch <- import("torch")

# Load pre-trained BERT model and tokenizer
tokenizer <- transformers$BertTokenizer$from_pretrained("bert-base-uncased")
model <- transformers$BertModel$from_pretrained("bert-base-uncased")
```

Transformers is a library developed by Hugging Face that provides pre-trained models and tools for NLP.

Word Embeddings - with BERT

```
library(reticulate)

# Use the desired Python environment (adjust the path to your specific environment)
use_python("C:/Users/Vito Giordano/anaconda3")

# Import transformers library
transformers <- import("transformers")
torch <- import("torch")

# Load pre-trained BERT model and tokenizer
tokenizer <- transformers$BertTokenizer$from_pretrained("bert-base-uncased")
model <- transformers$BertModel$from_pretrained("bert-base-uncased")
```

PyTorch is a machine learning library developed by Facebook that provides tools for deep learning and tensor computation.

Word Embeddings - with BERT

```
library(reticulate)

# Use the desired Python environment (adjust the path to your specific environment)
use_python("C:/Users/Vito Giordano/anaconda3")

# Import transformers library
transformers <- import("transformers")
torch <- import("torch")

# Load pre-trained BERT model and tokenizer
tokenizer <- transformers$BertTokenizer$from_pretrained("bert-base-uncased")
model <- transformers$BertModel$from_pretrained("bert-base-uncased")
```

bert-base-uncased is a pre-trained model from the BERT (Bidirectional Encoder Representations from Transformers) family.

Word Embeddings - with BERT



```
# Tokenize input text
input_text <- "King Queen Horse Donkey"
tokens <- tokenizer$encode_plus(
  input_text,
  add_special_tokens = TRUE,
  return_tensors = "pt"
)

# Convert tokens to tensors
input_ids <- tokens$input_ids
attention_mask <- tokens$attention_mask

# Get model output
with(torch$no_grad(), {
  outputs <- model(input_ids, attention_mask = attention_mask)
})
```

Tensors are multi-dimensional arrays that generalize scalars, vectors, and matrices to higher dimensions.

Word Embeddings - with BERT

```
# Extract the embeddings
embeddings <- outputs$last_hidden_state
print(embeddings)
```

```
## tensor([[-0.9114, -0.0300, -0.3343, ..., -0.3453, -0.0286,  0.0139],
##          [-0.1638, -0.0716,  0.1833, ..., -0.2992, -0.0586, -0.5135],
##          [ 0.2011, -0.0080,  0.5483, ..., -0.5800, -0.4435, -0.4137],
##          [ 0.5622,  0.2190,  0.3662, ..., -0.3214, -0.1649, -0.1823],
##          [-0.3408, -0.0891, -0.4561, ...,  0.0965,  0.1041, -0.0889],
##          [ 0.8376,  0.1417, -0.3342, ..., -0.0113, -0.6780, -0.2741]])
```

Word Embeddings - with BERT



Let's convert the embeddings from tensor to vectors.

```
# Get the tokens (words) from the input text
tokens_list <- tokenizer$tokenize(input_text)

# Convert the embeddings tensor to a numpy array
embeddings_np <- embeddings$numpy()

embedding_list <- list()
for (i in 1:length(tokens_list)) {
  embedding_list[[i]] <- embeddings_np[1, i+1, ]
}
```

Word Embeddings - with BERT

Then, let's create a dataframe to associate the embeddings to the corresponding token.

```
# Combine tokens and their embeddings into a dataframe
tokens_df <- data.frame(
  token = tokens_list
)

tokens_df <- tokens_df |>
  mutate(embeddings = embedding_list)
```

Word Embeddings - with BERT

Now we can use the embeddings to calculate the similarity between tokens.

```
embeddings_mat <- matrix(  
  unlist(tokens_df$embeddings),  
  ncol = 768, byrow = TRUE)  
  
embeddings_similarity <- embeddings_mat / sqrt(rowSums(embeddings_mat * embeddings_mat))  
embeddings_similarity <- embeddings_similarity %*% t(embeddings_similarity)  
  
embeddings_similarity <- embeddings_similarity |>  
  as_tibble()
```

Word Embeddings - with BERT

King and Queen are similar, and different from Donkey and Horse. The two animals are also similar between each other.

```
tokens_df <- data.frame(  
  token = tokens_list,  
  king = embeddings_similarity$V1,  
  queen = embeddings_similarity$V2,  
  horse = embeddings_similarity$V3,  
  donkey = embeddings_similarity$V4  
)
```

```
tokens_df
```

##	token	king	queen	horse	donkey
## 1	king	1.0000000	0.7577016	0.6678385	0.5803093
## 2	queen	0.7577016	1.0000000	0.7843059	0.5533188
## 3	horse	0.6678385	0.7843059	1.0000000	0.6974890
## 4	donkey	0.5803093	0.5533188	0.6974890	1.0000000

